

PM - repetition - no_frac - Serial position analysis

```
# do this in calling R script
# library(ggplot2)
# library(fmsb) # for TestModels
# library(lme4)
# library(kableExtra)
# library(MASS) # for dropterm
# library(tidyverse)
# library(dominanceanalysis)
# library(cowplot) # for multiple plots in one figure
# library(formatters) # to wrap strings for plot titles
# library(pals) # for continuous color palettes

# load needed functions
# source(paste0(RootDir, "/src/function_library/sp_functions.R"))
# do this in the calling R script

# for testing
# CurPat <- "GM"
# CurTask <- "repetition"
# MinLength <- 4
# MaxLength <- 10
# BestModelIndexL1 <- 1
# BestModelIndexL2 <- 1
# BestModelIndexL3 <- 1
# ReportTitle <- paste(CurPat, " - ", CurTask, " - ", RunLabel, " - Serial position analysis")
# RandomSamples <- 10
# DoSimulations <- FALSE
# RemoveFracPres <- TRUE

CurPat <- params$CurPat
CurTask <- params$CurTask
MinLength <- params$MinLength
MaxLength <- params$MaxLength
BestModelIndexL1 <- params$BestModelIndexL1
BestModelIndexL2 <- params$BestModelIndexL2
BestModelIndexL3 <- params$BestModelIndexL3
ReportTitle <- params$ReportTitle
RandomSamples <- params$RandomSamples
DoSimulations <- params$DoSimulations
RemoveFracPres <- params$RemoveFracPres

# done in calling script
# print(paste0("Using root dir: ", RootDir))
# RunDir <- paste0(paste0(RootDir, "/output/"), RunLabel))
# if(!dir.exists(RunDir)){
#   dir.create(RunDir, showWarnings = TRUE)
#   print(paste0("created run directory: ", RunDir))
# }
```

```

# # create patient directory if it doesn't already exist
# PatientDir <- paste0(RootDir, "/output/", RunLabel, "/", CurPat)
# if(!dir.exists(PatientDir)){
#   dir.create(PatientDir, showWarnings = TRUE)
#   print(paste0("created participant directory: ", PatientDir))
# }
# TablesDir <- paste0(RootDir, "/output/", RunLabel, "/", CurPat, "/tables/")
# if(!dir.exists(TablesDir)){
#   dir.create(TablesDir, showWarnings = TRUE)
#   print(paste0("created tables directory: ", TablesDir))
# }
# FigDir <- paste0(RootDir, "/output/", RunLabel, "/", CurPat, "/fig/")
# if(!dir.exists(FigDir)){
#   dir.create(FigDir, showWarnings = TRUE)
#   print(paste0("created figures directory: ", FigDir))
# }
# ReportsDir <- paste0(RootDir, "/output/", RunLabel, "/", CurPat, "/reports/")
# if(!dir.exists(ReportsDir)){
#   dir.create(ReportsDir, showWarnings = TRUE)
#   print(paste0("created reports directory: ", ReportsDir))
# }
results_report_DF <- data.frame(Description=character(), ParameterValue=double())

ModelDatFilename<-paste0(RootDir, "/data/", CurPat, "/", CurPat, "_", CurTask, "_posdat.csv")
if(!file.exists(ModelDatFilename)){
  print("**ERROR** Input data file not found")
  print(sprintf("\tfile: ", ModelDatFilename))
  print(sprintf("\troot dir: ", RootDir))
}
PosDat<-read.csv(ModelDatFilename)

# may already be done in datafile
# if(RemoveFinalPosition){
#   PosDat <- PosDat[PosDat$pos < PosDat$stimlen,] # remove final position
# }
PosDat <- PosDat[PosDat$stimlen >= MinLength,] # include only lengths with sufficient N
PosDat <- PosDat[PosDat$stimlen <= MaxLength,]
# include only correct and nonword errors (not no response, word or w+nw errors)
PosDat <- PosDat[(PosDat$err_type == "correct") | (PosDat$err_type == "nonword"), ]
PosDat <- PosDat[(PosDat$pos) != (PosDat$stimlen), ]
# make sure final positions have been removed
PosDat$stim_number <- NumberStimuli(PosDat)
PosDat$raw_log_freq <- PosDat$log_freq
PosDat$log_freq <- PosDat$log_freq - mean(PosDat$log_freq) # centre frequency
OrigDataLen <- nrow(PosDat)
print(sprintf ("Original data has %d values", OrigDataLen))

## [1] "Original data has 3401 values"

if(RemoveFracPres){ # eliminate fractional preserved values?
  PosDat <- PosDat[(PosDat$preserved == 0) | (PosDat$preserved == 1),]
  DataLen<- nrow(PosDat)
  print(sprintf("Data without fractional preserved values has %d values", DataLen))
  print(sprintf("%d values eliminated.", OrigDataLen - DataLen))
}

```

```
## [1] "Data without fractional preserved values has 3201 values"
## [1] "200 values eliminated."
```

```
PosDat$CumPres <- CalcCumPres(PosDat)
PosDat$CumErr <- CalcCumErrFromPreserved(PosDat)
```

```
# plot distribution of complex onsets and codas
# across serial position in the stimuli -- do complexities concentrate
# on one part of the serial position curve?
# syll_component = 1 is coda
# syll_component = S is satellite
```

```
# get counts of syllable components in different serial positions
```

```
syll_comp_dist_N <- PosDat %>% mutate(pos_factor = as.factor(pos), syll_component_factor = as.factor(syll_component))
```

```
syll_comp_dist_perc <- syll_comp_dist_N %>% ungroup() %>%
  mutate(across(O:S, ~ . / total))
```

```
kable(syll_comp_dist_N)
```

pos_factor	O	P	V	1	S	total
1	435	26	108	NA	NA	569
2	56	NA	347	69	86	558
3	239	NA	140	153	12	544
4	224	NA	181	47	27	479
5	165	NA	148	55	34	402
6	145	NA	85	43	17	290
7	112	NA	63	22	16	213
8	62	NA	24	15	3	104
9	36	NA	1	NA	5	42

```
kable(syll_comp_dist_perc)
```

pos_factor	O	P	V	1	S	total
1	0.7644991	0.0456942	0.1898067	NA	NA	569
2	0.1003584	NA	0.6218638	0.1236559	0.1541219	558
3	0.4393382	NA	0.2573529	0.2812500	0.0220588	544
4	0.4676409	NA	0.3778706	0.0981211	0.0563674	479
5	0.4104478	NA	0.3681592	0.1368159	0.0845771	402
6	0.5000000	NA	0.2931034	0.1482759	0.0586207	290
7	0.5258216	NA	0.2957746	0.1032864	0.0751174	213
8	0.5961538	NA	0.2307692	0.1442308	0.0288462	104
9	0.8571429	NA	0.0238095	NA	0.1190476	42

```
# get percentages only from summary table
```

```
syll_comp_perc <- syll_comp_dist_perc[,2:(ncol(syll_comp_dist_perc)-1)]
```

```
syll_comp_perc$pos <- as.integer(as.character(syll_comp_dist_perc$pos_factor))
```

```
syll_comp_perc <- syll_comp_perc %>% rename("Coda" = "1", "Satellite" = "S")
```

```
# pivot to long format for plotting
```

```
syll_comp_plot_data <- syll_comp_perc %>% pivot_longer(cols=seq(1:(ncol(syll_comp_perc)-1)),
  names_to="syll_component", values_to="percent") %>% filter(syll_component %in% c("Coda", "Satellite"))
```

```
# plot satellite and coda occurrences across position

syll_comp_plot <- ggplot(syll_comp_plot_data,
                        aes(x=pos,y=percent,group=syll_component,
                           color=syll_component,
                           linetype = syll_component,
                           shape = syll_component)) +
  geom_line() + geom_point() + scale_color_manual(name="Syllable component", values = palette_values) +
syll_comp_plot <- syll_comp_plot + scale_y_continuous(name="Percent of segment types") + scale_linetype_manual(name="Syllable component", values = c("solid", "dashed")) +
  scale_shape_discrete(name="Syllable component", values = c("circle", "triangle"))
ggsave(paste0(FigDir, CurPat, "_", RunLabel, "_", CurTask, "_pos_of_syll_complexities_in_stimuli.png"), plot=syll_comp_plot)
```

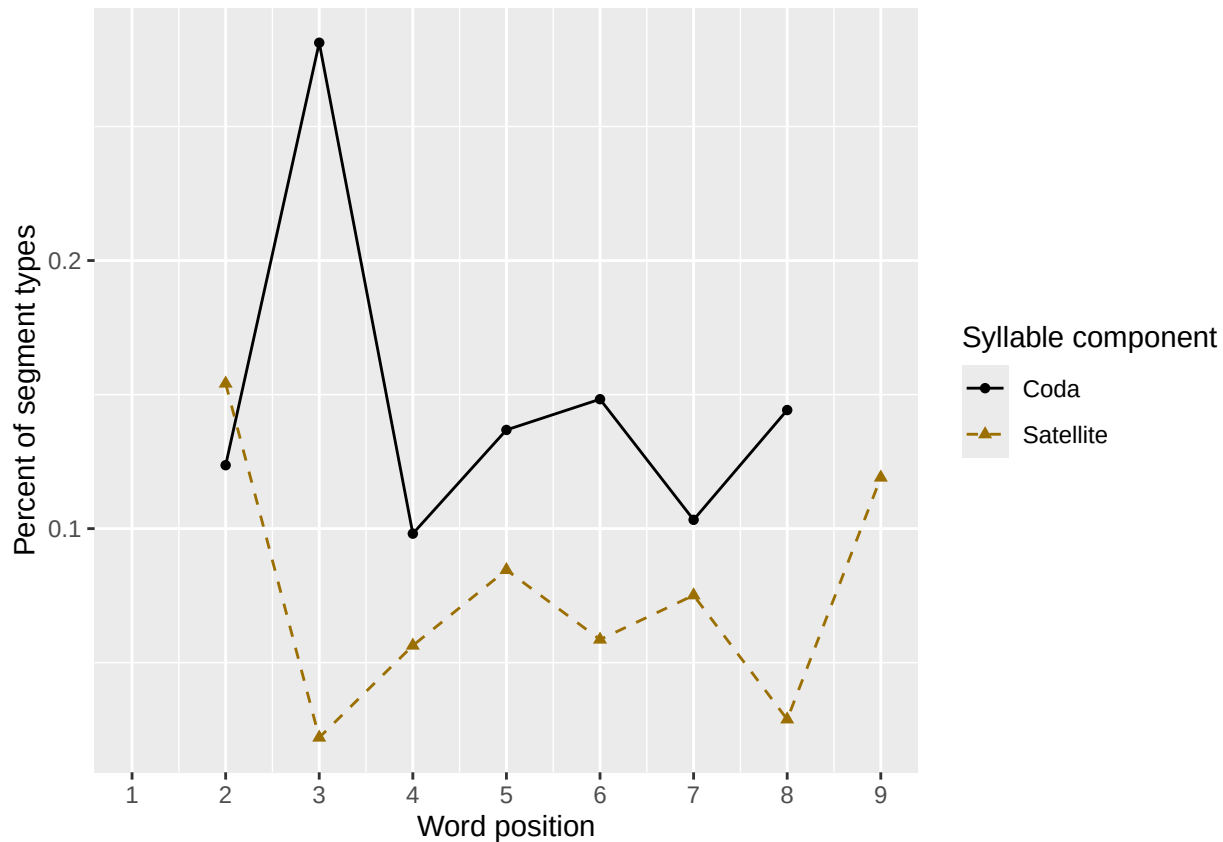
Warning: Removed 3 rows containing missing values or values outside the scale range (``geom_line()``).

Warning: Removed 3 rows containing missing values or values outside the scale range (``geom_point()``).

syll_comp_plot

```
## Warning: Removed 3 rows containing missing values or values outside the scale range (`geom_line()`).
```

```
## Removed 3 rows containing missing values or values outside the scale range (`geom_point()`).
```



```
# len/pos table
pos_len_summary <- PosDat %>% group_by(stimlen,pos) %>% summarise(preserved = mean(preserved))

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.

min_preserved_raw <- min(as.numeric(unlist(pos_len_summary$preserved)))
max_preserved_raw <- max(as.numeric(unlist(pos_len_summary$preserved)))
preserved_range <- (max_preserved_raw - min_preserved_raw)
```

```

min_preserved <- min_preserved_raw - (0.1 * preserved_range)
max_preserved <- max_preserved_raw + (0.1 * preserved_range)
pos_len_table <- pos_len_summary %>% pivot_wider(names_from = pos, values_from = preserved)
write.csv(pos_len_table, paste0(TablesDir, CurPat, "_", RunLabel, "_", CurTask, "_preserved_percentages_by_len",
pos_len_table

```

```

## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`   `4`   `5`   `6`   `7`   `8`   `9`
##   <int> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1      4 0.821 0.929 0.857 NA      NA      NA      NA      NA      NA
## 2      5 0.951 0.829 0.875 0.9   NA      NA      NA      NA      NA
## 3      6 0.913 0.856 0.81  0.902 0.81    NA      NA      NA      NA
## 4      7 0.948 0.926 0.872 0.772 0.826   0.802 NA      NA      NA
## 5      8 0.930 0.912 0.862 0.811 0.807   0.83   0.738 NA      NA
## 6      9 0.821 0.875 0.75  0.679 0.754   0.75   0.656 0.721 NA
## 7     10 0.667 0.907 0.805 0.744 0.75    0.860 0.711 0.791 0.690

```

```

# len/pos table
pos_len_N <- PosDat %>% group_by(stimlen, pos) %>% summarise(preserved = mean(preserved), N=n()) %>% dplyr::

```

`summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.

```

pos_len_N_table <- pos_len_N %>% pivot_wider(names_from = pos, values_from = N)
write.csv(pos_len_N_table, paste0(TablesDir, CurPat, "_", RunLabel, "_", CurTask,
"_N_by_len_position.csv"))
pos_len_N_table

```

```

## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`   `4`   `5`   `6`   `7`   `8`   `9`
##   <int> <int> <int> <int> <int> <int> <int> <int> <int> <int>
## 1      4    56    56    56    NA    NA    NA    NA    NA    NA
## 2      5    82    82    80    80    NA    NA    NA    NA    NA
## 3      6   104   104   100   102   100    NA    NA    NA    NA
## 4      7    97    95    94    92    92    91    NA    NA    NA
## 5      8   115   114   109   106   109   100   107    NA    NA
## 6      9    67    64    64    56    57    56    61    61    NA
## 7     10    48    43    41    43    44    43    45    43    42

```

```

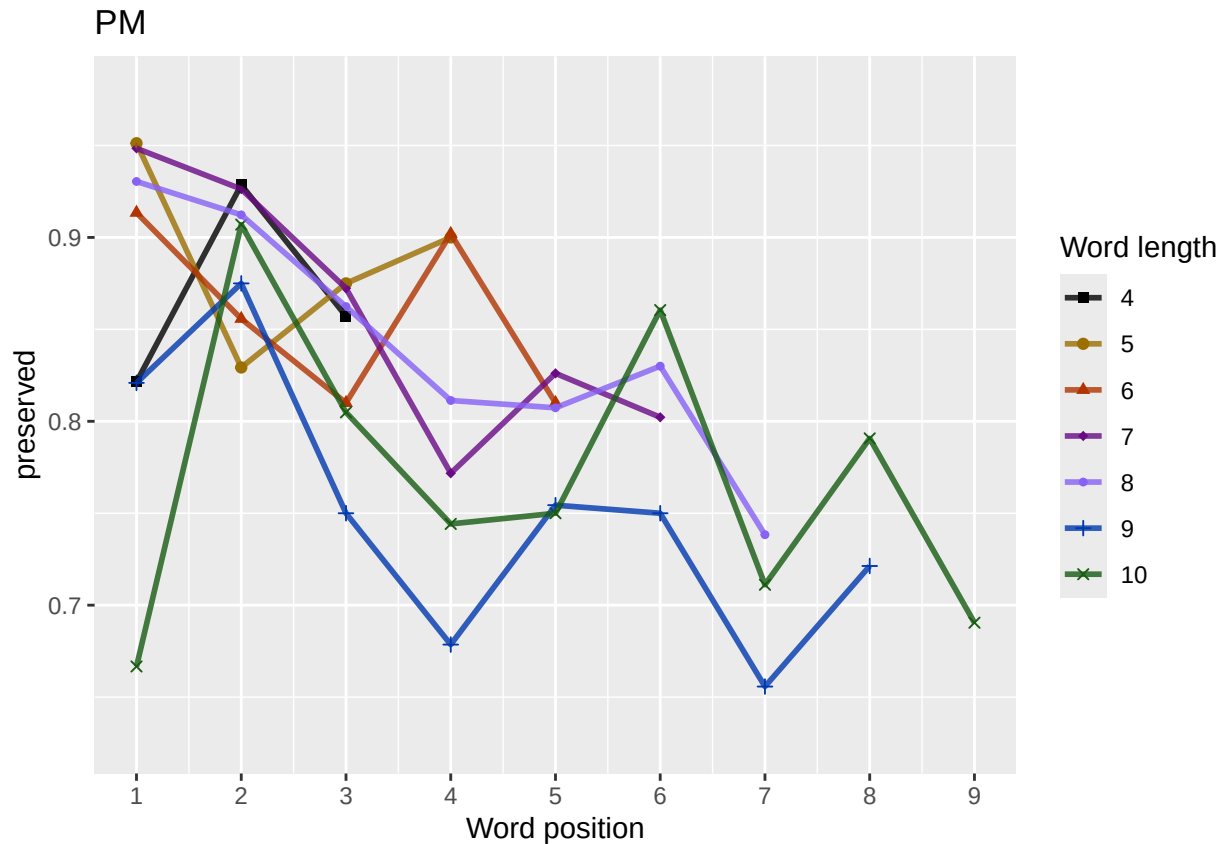
obs_linetypes <- c("solid", "solid", "solid", "solid",
"solid", "solid", "solid", "solid", "solid")
pred_linetypes <- "dashed"
pos_len_summary$stimlen <- factor(pos_len_summary$stimlen)
pos_len_summary$pos <- factor(pos_len_summary$pos)
# len_pos_plot <- ggplot(pos_len_summary, aes(x=pos, y=preserved, group=stimlen, shape=stimlen, color=stimlen))
len_pos_plot <- plot_len_pos_obs_predicted(PosDat,
paste0(CurPat),
NULL,
c(min_preserved, max_preserved),
palette_values,
shape_values,
obs_linetypes,
pred_linetypes)

```

`summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## Scale for y is already present. Adding another scale for y, which will replace the existing scale.
```

```
ggsave(paste0(FigDir, CurPat, "_", RunLabel, "_", CurTask,
              "_percent_preserved_by_length_pos.png"),
       plot=len_pos_plot, device="png", unit="cm", width=15, height=11)
len_pos_plot
```



Length and position

```
# length and position
```

```
LPMModelEquations<-c("preserved ~ 1",
  "preserved ~ stimlen",
  "preserved ~ pos",
  "preserved ~ stimlen + pos",
  "preserved ~ stimlen*pos",
  "preserved ~ I(pos^2)+pos",
  "preserved ~ stimlen + I(pos^2) + pos",
  "preserved ~ stimlen * (I(pos^2) + pos)"
)
```

```
LPres<-TestModels(LPMModelEquations,PosDat)
```

```
## *****
```

```
## model index: 4
```

```
##
```

```
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
```

```
## data = PosDat)
```

```

##
## Coefficients:
## (Intercept)      stimlen      pos
##      2.8920      -0.1040      -0.1313
##
## Degrees of Freedom: 3200 Total (i.e. Null);  3198 Residual
## Null Deviance:      2902
## Residual Deviance: 2836 AIC: 2842
## log likelihood: -1418.173
## Nagelkerke R2:  0.03412193
## % pres/err predicted correctly: -876.5652
## % of predictable range [ (model-null)/(1-null) ]:  0.02110016
## *****
## model index:  7
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      I(pos^2)      pos
##      3.14443      -0.11254      0.01329      -0.24621
##
## Degrees of Freedom: 3200 Total (i.e. Null);  3197 Residual
## Null Deviance:      2902
## Residual Deviance: 2835 AIC: 2843
## log likelihood: -1417.32
## Nagelkerke R2:  0.03499856
## % pres/err predicted correctly: -876.375
## % of predictable range [ (model-null)/(1-null) ]:  0.02131229
## *****
## model index:  5
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      pos  stimlen:pos
##      3.33583      -0.15846      -0.26601      0.01599
##
## Degrees of Freedom: 3200 Total (i.e. Null);  3197 Residual
## Null Deviance:      2902
## Residual Deviance: 2835 AIC: 2843
## log likelihood: -1417.635
## Nagelkerke R2:  0.0346748
## % pres/err predicted correctly: -876.4329
## % of predictable range [ (model-null)/(1-null) ]:  0.02124772
## *****
## model index:  8
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
##      (Intercept)      stimlen      I(pos^2)      pos  stimlen:I(pos^2)  stiml

```

```

##          3.243388          -0.128748          0.009200          -0.262066          0.000197          0.0
##
## Degrees of Freedom: 3200 Total (i.e. Null);  3195 Residual
## Null Deviance:          2902
## Residual Deviance: 2835 AIC: 2847
## log likelihood: -1417.275
## Nagelkerke R2:  0.0350441
## % pres/err predicted correctly: -876.3618
## % of predictable range [ (model-null)/(1-null) ]:  0.02132702
## *****
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##          data = PosDat)
##
## Coefficients:
## (Intercept)          pos
##      2.2428      -0.1662
##
## Degrees of Freedom: 3200 Total (i.e. Null);  3199 Residual
## Null Deviance:          2902
## Residual Deviance: 2847 AIC: 2851
## log likelihood: -1423.37
## Nagelkerke R2:  0.02877735
## % pres/err predicted correctly: -879.4734
## % of predictable range [ (model-null)/(1-null) ]:  0.0178562
## *****
## model index:  6
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##          data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)          pos
##      2.339295      0.006442      -0.223211
##
## Degrees of Freedom: 3200 Total (i.e. Null);  3198 Residual
## Null Deviance:          2902
## Residual Deviance: 2846 AIC: 2852
## log likelihood: -1423.162
## Nagelkerke R2:  0.02899198
## % pres/err predicted correctly: -879.5106
## % of predictable range [ (model-null)/(1-null) ]:  0.01781467
## *****
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##          data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      2.9018      -0.1728
##
## Degrees of Freedom: 3200 Total (i.e. Null);  3199 Residual

```

```
## Null Deviance:      2902
## Residual Deviance: 2866 AIC: 2870
## log likelihood: -1432.973
## Nagelkerke R2: 0.01885498
## % pres/err predicted correctly: -885.1918
## % of predictable range [ (model-null)/(1-null) ]: 0.01147747
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)
## 1.597
##
## Degrees of Freedom: 3200 Total (i.e. Null); 3200 Residual
## Null Deviance:      2902
## Residual Deviance: 2902 AIC: 2904
## log likelihood: -1451.064
## Nagelkerke R2: 0
## % pres/err predicted correctly: -895.4811
## % of predictable range [ (model-null)/(1-null) ]: 0
## *****
```

```
BestLPModel<-LPRes$ModelResult[[1]]
BestLPModelFormula<-LPRes$Model[[1]]

LPAICSummary<-data.frame(Model=LPRes$Model,
                          AIC=LPRes$AIC,
                          row.names = LPRes$Model)
LPAICSummary$DeltaAIC<-LPAICSummary$AIC-LPAICSummary$AIC[1]
LPAICSummary$AICexp<-exp(-0.5*LPAICSummary$DeltaAIC)
LPAICSummary$AICwt<-LPAICSummary$AICexp/sum(LPAICSummary$AICexp)
LPAICSummary$NagR2<-LPRes$NagR2

LPAICSummary <- merge(LPAICSummary,LPRes$CoefficientValues, by='row.names',sort=FALSE)
LPAICSummary <- subset(LPAICSummary, select = -c(Row.names))

write.csv(LPAICSummary,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                              "_len_pos_model_summary.csv"),row.names = FALSE)

kable(LPAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	stimlen	pos	stimlen:I(pos)	I(pos^2)	stimlen:I(pos^2)
preserved ~ stimlen + pos	2842.347	0.000000	0.000000	0.037898	0.8303412	29891999	-	-	NA	NA	NA
preserved ~ stimlen + I(pos^2)	2842.639	0.292061	0.186413	0.332749	0.5703499	86144432	-	-	NA	0.0132897	NA
+ pos							0.1125385	2462129			
preserved ~ stimlen * pos	2843.270	0.922947	0.630350	0.123889	0.6903467	43335830	-	-	0.0159942	NA	NA
preserved ~ stimlen * (I(pos^2)	2846.550	4.203315	3.122259	0.046332	0.2703504	41243388	-	-	0.0043390	0.0091995	0.000197
+ pos)							0.1287476	2620656			

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	stimlen	pos	stimlen:I(pos)	I(pos^2)	stimlen:I(pos^2)
preserved ~ pos	2850.748	0.3932561	0.1015046	0.2005702	0.2302877	23242829	NA	-	NA	NA	NA
								0.1661569			
preserved ~ I(pos^2) + pos	2852.323	1.9765209	0.0068107	0.0025838	0.2899203	39295	NA	-	NA	0.0064425	NA
								0.2232112			
preserved ~ stimlen	2869.947	7.5995866	0.0000000	0.0000000	0.0188530	901848	-	NA	NA	NA	NA
								0.1728231			
preserved ~ 1	2904.129	51.7818666	0.0000000	0.0000000	0.0000000	597117	NA	NA	NA	NA	NA

```
print(BestLPModelFormula)
```

```
## [1] "preserved ~ stimlen + pos"
```

```
print(BestLPModel)
```

```
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos
##      2.8920      -0.1040      -0.1313
##
## Degrees of Freedom: 3200 Total (i.e. Null);  3198 Residual
## Null Deviance:      2902
## Residual Deviance: 2836  AIC: 2842
```

```
PosDat$LPFitted<-fitted(BestLPModel)
```

```
fitted_len_pos_plot <- plot_len_pos_obs_predicted(PosDat, PosDat$patient[1],
NULL,palette_values=palette_values)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
# len/pos table
```

```
fitted_pos_len_summary <- PosDat %>% group_by(stimlen,pos) %>% summarise(fitted = mean(LPfitted))
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
fitted_pos_len_table <- fitted_pos_len_summary %>% pivot_wider(names_from = pos, values_from = fitted)
fitted_pos_len_table
```

```
## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`   `4`   `5`   `6`   `7`   `8`   `9`
##   <int> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1     4  0.912  0.901  0.889 NA     NA     NA     NA     NA     NA
## 2     5  0.904  0.892  0.878  0.864 NA     NA     NA     NA     NA
## 3     6  0.894  0.881  0.867  0.851  0.834 NA     NA     NA     NA
## 4     7  0.884  0.870  0.854  0.837  0.819  0.798 NA     NA     NA
## 5     8  0.873  0.858  0.841  0.823  0.803  0.781  0.758 NA     NA
## 6     9  0.861  0.845  0.827  0.807  0.786  0.763  0.738  0.712 NA
## 7    10  0.848  0.830  0.811  0.790  0.768  0.743  0.718  0.690  0.662
```

```
fitted_pos_len_summary$stimlen<-factor(fitted_pos_len_summary$stimlen)
```

```
fitted_pos_len_summary$pos<-factor(fitted_pos_len_summary$pos)
```

```

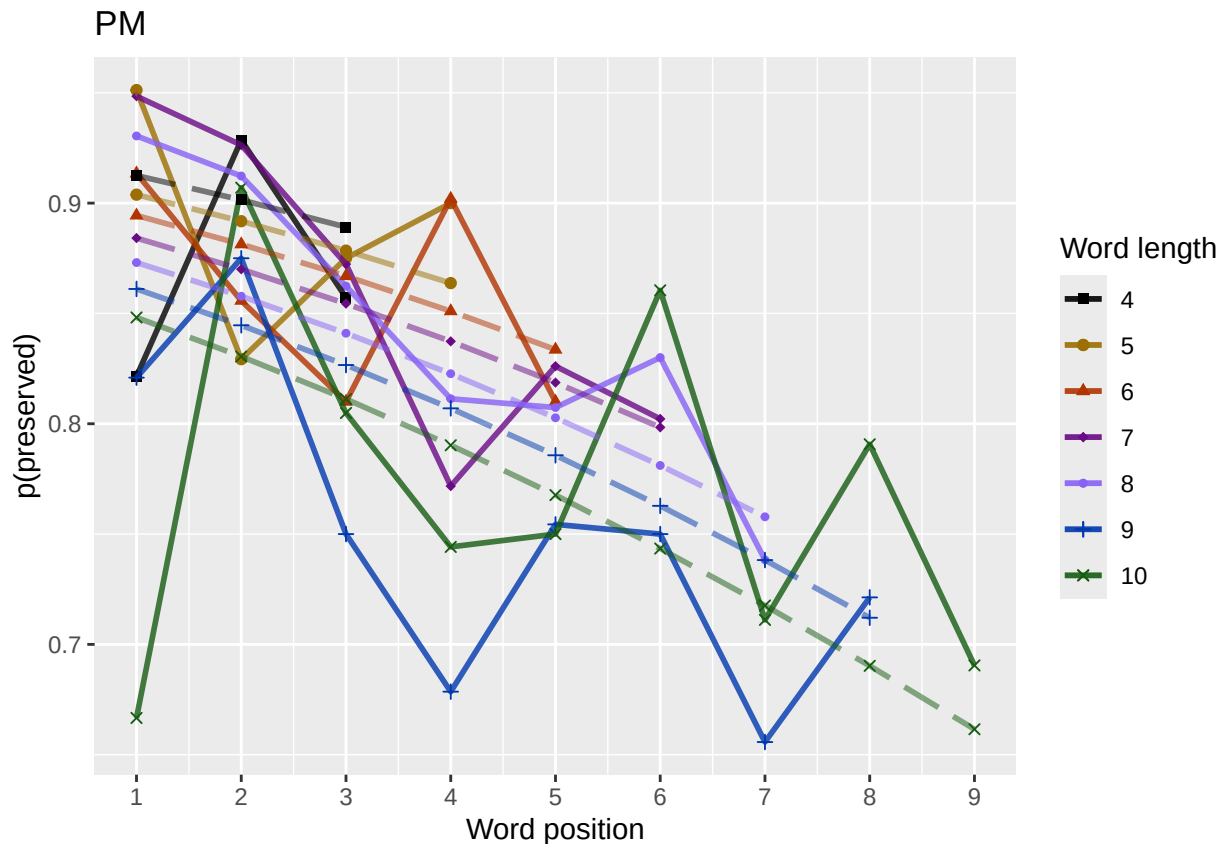
# fitted_len_pos_plot <- ggplot(pos_len_summary, aes(x=pos, y=preserved, group=stimlen, shape=stimlen, color=
# fitted_len_pos_plot <- fitted_len_pos_plot + geom_line(data=fitted_pos_len_summary, aes(x=pos, y=fitted
# geom_point(data=fitted_pos_len_summary, aes(x=pos, y=fitted, group=stimlen, shape=stimlen))) + ggtitle(pas

fitted_len_pos_plot <- plot_len_pos_obs_predicted(PosDat,
  paste0(PosDat$patient[1]),
  "LPFitted",
  NULL,
  palette_values,
  shape_values,
  obs_linetypes,
  pred_linetypes = c("longdash")
)

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.

ggsave(paste0(FigDir, CurPat, "_", RunLabel, "_", CurTask, "_percent_preserved_by_length_pos_wfit.png"), plot=fitted_len_pos_plot

```



length and position without fragments to see if this changes position² influence

```

# first number responses, then count resp with fragments - below we will eliminate fragments
# and re-run models

# number responses
resp_num <- 0

```

```

prev_pos<-9999 # big number to initialize (so first position is smaller)
resp_num_array <- integer(length = nrow(PosDat))
for(i in seq(1,nrow(PosDat))){
  if(PosDat$pos[i] <= prev_pos){ # equal for resp length 1
    resp_num <- resp_num + 1
  }
  resp_num_array[i]<-resp_num
  prev_pos <- PosDat$pos[i]
}
PosDat$resp_num <- resp_num_array

# count responses with fragments
resp_with_frag <- PosDat %>% group_by(resp_num) %>% summarise(frag = as.logical(sum(fragment)))
num_frag <- resp_with_frag %>% summarise(frag_sum = sum(frag), N=n())
num_frag

## # A tibble: 1 x 2
##   frag_sum      N
##   <int> <int>
## 1      10    573

num_frag$percent_with_frag[1] <- (num_frag$frag_sum[1] / num_frag$N[1])*100

## Warning: Unknown or uninitialised column: `percent_with_frag`.

print(sprintf("The number of responses with fragments was %d / %d = %.2f percent",
  num_frag$frag_sum[1],num_frag$N[1],num_frag$percent_with_frag[1]))

## [1] "The number of responses with fragments was 10 / 573 = 1.75 percent"

write.csv(num_frag,paste0(TablesDir,CurPat,"_",RunLabel,"_",
  CurTask,"_percent_fragments.csv"),row.names = FALSE)

# length and position models for data without fragments
NoFragData <- PosDat %>% filter(fragment != 1)

NoFrag_LPRes<-TestModels(LPModelEquations,NoFragData)

## *****
## model index: 7
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##   data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      I(pos^2)          pos
##   3.19231      -0.11328      0.01971      -0.28249
##
## Degrees of Freedom: 3183 Total (i.e. Null);  3180 Residual
## Null Deviance:      2841
## Residual Deviance: 2786 AIC: 2794
## log likelihood:  -1393.024
## Nagelkerke R2:  0.02904289
## % pres/err predicted correctly:  -856.7916
## % of predictable range [ (model-null)/(1-null) ]:  0.01724265
## *****

```

```

## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos
##      2.8271      -0.1012      -0.1137
##
## Degrees of Freedom: 3183 Total (i.e. Null); 3181 Residual
## Null Deviance:      2841
## Residual Deviance: 2790 AIC: 2796
## log likelihood: -1394.792
## Nagelkerke R2: 0.02719185
## % pres/err predicted correctly: -857.5244
## % of predictable range [ (model-null)/(1-null) ]: 0.01640305
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos stimlen:pos
##      3.39877      -0.17141      -0.28915      0.02089
##
## Degrees of Freedom: 3183 Total (i.e. Null); 3180 Residual
## Null Deviance:      2841
## Residual Deviance: 2788 AIC: 2796
## log likelihood: -1393.908
## Nagelkerke R2: 0.0281182
## % pres/err predicted correctly: -857.1702
## % of predictable range [ (model-null)/(1-null) ]: 0.01680893
## *****
## model index: 8
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      I(pos^2)          pos stimlen:I(pos^2)      stimlen:I(pos^2)
##      3.185910      -0.116098      0.008267      -0.235954      0.001084      -0.001084
##
## Degrees of Freedom: 3183 Total (i.e. Null); 3178 Residual
## Null Deviance:      2841
## Residual Deviance: 2786 AIC: 2798
## log likelihood: -1392.984
## Nagelkerke R2: 0.02908382
## % pres/err predicted correctly: -856.7669
## % of predictable range [ (model-null)/(1-null) ]: 0.017271
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",

```

```

##      data = PosDat)
##
## Coefficients:
## (Intercept)          pos
##      2.1946      -0.1472
##
## Degrees of Freedom: 3183 Total (i.e. Null);  3182 Residual
## Null Deviance:      2841
## Residual Deviance: 2799  AIC: 2803
## log likelihood:  -1399.666
## Nagelkerke R2:  0.02208137
## % pres/err predicted correctly:  -860.1934
## % of predictable range [ (model-null)/(1-null) ]:  0.01334522
## *****
## model index:  6
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)          pos
##      2.3807      0.0127      -0.2585
##
## Degrees of Freedom: 3183 Total (i.e. Null);  3181 Residual
## Null Deviance:      2841
## Residual Deviance: 2798  AIC: 2804
## log likelihood:  -1398.904
## Nagelkerke R2:  0.02288179
## % pres/err predicted correctly:  -859.9554
## % of predictable range [ (model-null)/(1-null) ]:  0.01361797
## *****
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      2.8285      -0.1593
##
## Degrees of Freedom: 3183 Total (i.e. Null);  3182 Residual
## Null Deviance:      2841
## Residual Deviance: 2811  AIC: 2815
## log likelihood:  -1405.526
## Nagelkerke R2:  0.01591582
## % pres/err predicted correctly:  -863.4709
## % of predictable range [ (model-null)/(1-null) ]:  0.009590248
## *****
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:

```

```

## (Intercept)
##      1.629
##
## Degrees of Freedom: 3183 Total (i.e. Null);  3183 Residual
## Null Deviance:      2841
## Residual Deviance: 2841  AIC: 2843
## log likelihood:  -1420.553
## Nagelkerke R2:   1.880807e-16
## % pres/err predicted correctly:  -871.8417
## % of predictable range [ (model-null)/(1-null) ]:  0
## *****

NoFragBestLPModel<-NoFrag_LPRes$ModelResult[[1]]
NoFragBestLPModelFormula<-NoFrag_LPRes$Model[[1]]

NoFragLPAICSummary<-data.frame(Model=NoFrag_LPRes$Model,
                                AIC=NoFrag_LPRes$AIC,
                                row.names = NoFrag_LPRes$Model)
NoFragLPAICSummary$DeltaAIC<-NoFragLPAICSummary$AIC-NoFragLPAICSummary$AIC[1]
NoFragLPAICSummary$AICexp<-exp(-0.5*NoFragLPAICSummary$DeltaAIC)
NoFragLPAICSummary$AICwt<-NoFragLPAICSummary$AICexp/sum(NoFragLPAICSummary$AICexp)
NoFragLPAICSummary$NagR2<-NoFrag_LPRes$NagR2

NoFragLPAICSummary <- merge(NoFragLPAICSummary,NoFrag_LPRes$CoefficientValues, by='row.names',sort=FALSE)
NoFragLPAICSummary <- subset(NoFragLPAICSummary, select = -c(Row.names))

write.csv(NoFragLPAICSummary,paste0(TablesDir,
                                    CurPat,"_",RunLabel,"_",
                                    CurTask,
                                    "_no_fragments_len_pos_model_summary.csv"),
          row.names = FALSE)
kable(NoFragLPAICSummary)

```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	stimlen	pos	stimlen:I(pos^2)	stimlen:I(pos^2)
preserved ~ stimlen + I(pos^2) + pos	2794.047	0.000000	1.000000	0.0491468	0.290429	192310	-	-	NA	0.0197104 NA
preserved ~ stimlen + pos	2795.585	1.537716	0.463542	0.1227816	0.3027192	2827059	-	-	NA	NA NA
preserved ~ stimlen * pos	2795.815	1.767748	0.413179	0.1203064	0.5028118	2398768	-	-	0.0208921	NA NA
preserved ~ stimlen * (I(pos^2) + pos)	2797.963	3.921725	0.140737	0.0069167	0.8029083	38185910	-	-	-	0.00826710010842
preserved ~ pos	2803.332	9.284541	0.009635	0.8004735	0.7022082	24194612	NA	-	NA	NA NA
preserved ~ I(pos^2) + pos	2803.807	9.759932	0.007597	0.3003733	0.8022882	28380680	NA	-	NA	0.0127046 NA
preserved ~ stimlen	2815.052	11.004047	0.000027	0.5000013	0.5015912	2828490	-	NA	NA	NA NA
preserved ~ 1	2843.106	49.058998	0.000000	0.0000000	0.0000000	0629165	NA	NA	NA	NA NA

```
# plot no fragment data
```

```
NoFragData$LPFitted<-fitted(NoFragBestLPModel)
nofrag_obs_len_pos_plot <- plot_len_pos_obs_predicted(NoFragData, NoFragData$patient[1],
  NULL,palette_values=palette_values)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
# len/pos table
```

```
nofrag_fitted_pos_len_summary <- NoFragData %>% group_by(stimlen,pos) %>% summarise(fitted = mean(LPfit
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
nofrag_fitted_pos_len_table <- nofrag_fitted_pos_len_summary %>% pivot_wider(names_from = pos, values_f
nofrag_fitted_pos_len_table
```

```
## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`   `4`   `5`   `6`   `7`   `8`   `9`
##   <int> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1      4 0.922 0.905 0.888 NA      NA      NA      NA      NA      NA
## 2      5 0.914 0.895 0.876 0.860 NA      NA      NA      NA      NA
## 3      6 0.905 0.884 0.863 0.845 0.831 NA      NA      NA      NA
## 4      7 0.894 0.871 0.849 0.830 0.815 0.804 NA      NA      NA
## 5      8 0.883 0.858 0.834 0.813 0.797 0.786 0.781 NA      NA
## 6      9 0.871 0.844 0.818 0.795 0.778 0.766 0.762 0.764 NA
## 7     10 0.858 0.828 0.800 0.776 0.758 0.745 0.740 0.743 0.753
```

```
fitted_pos_len_summary$stimlen<-factor(nofrag_fitted_pos_len_summary$stimlen)
```

```
fitted_pos_len_summary$pos<-factor(nofrag_fitted_pos_len_summary$pos)
```

```
# fitted_len_pos_plot <- ggplot(pos_len_summary,aes(x=pos,y=preserved,group=stimlen,shape=stimlen,color
```

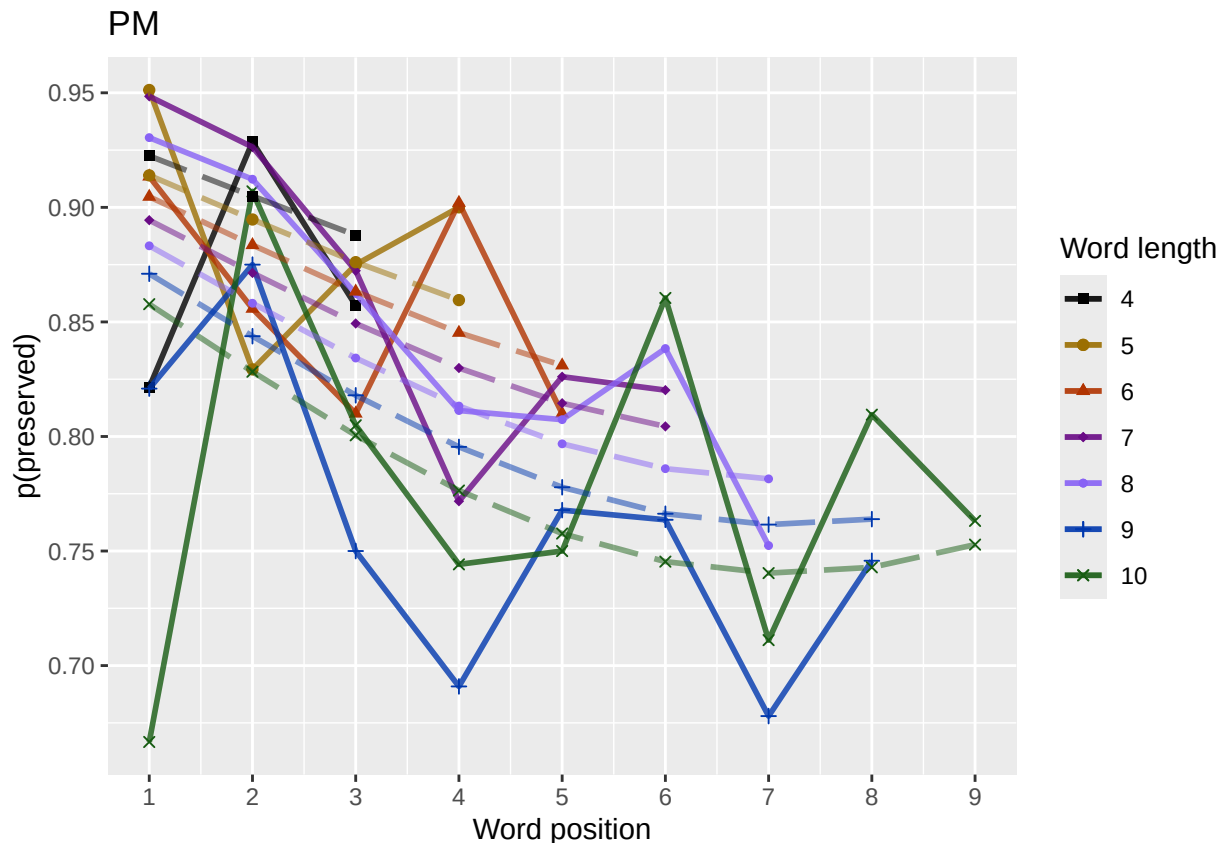
```
# fitted_len_pos_plot <- fitted_len_pos_plot + geom_line(data=fitted_pos_len_summary, aes(x=pos,y=fitted
```

```
# geom_point(data=fitted_pos_len_summary,aes(x=pos,y=fitted,group=stimlen,shape=stimlen)) + ggtitle(pas
```

```
nofrag_fitted_len_pos_plot <- plot_len_pos_obs_predicted(NoFragData,
  paste0(NoFragData$patient[1]),
  "LPFitted",
  NULL,
  palette_values,
  shape_values,
  obs_linetypes,
  pred_linetypes = c("longdash")
)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
ggsave(paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,
  "no_fragments_percent_preserved_by_length_pos_wfit.png"),
  plot=nofrag_fitted_len_pos_plot,
  device="png",unit="cm",
  width=15,height=11)
nofrag_fitted_len_pos_plot
```



back to full data

```
results_report_DF <- AddReportLine(results_report_DF,"min preserved",min_preserved)
results_report_DF <- AddReportLine(results_report_DF,"max preserved",max_preserved)
print(sprintf("Min/max preserved range: %.2f - %.2f",min_preserved,max_preserved))
```

```
## [1] "Min/max preserved range: 0.63 - 0.98"
```

```
# find the average difference between lengths and the average difference
# between positions taking the other factor into account
```

```
# choose fitted or raw -- we use fitted values because they are smoothed averages
# that take into account the influence of the other variable
```

```
table_to_use <- fitted_pos_len_table
```

```
# take away column with length information
```

```
table_to_use <- table_to_use[,2:ncol(table_to_use)]
```

```
# take averages for positions
```

```
# don't want downward estimates influenced by return upward of U
```

```
# therefore, for downward influence, use only the values before the min
```

```
# take the difference between each value (differences between position proportion correct) **NOTE** pro
```

```
# average the difference in probabilities
```

```
# in case min is close to the end or we are not using a min (for non-U-shaped)
```

```
# functions, we need to get differences _first_ and then average those (e.g.
```

```
# if there is an effect of length and we just average position data,
```

```
# later positions) will have a lower average (because of the length effect)
```

```
# even if all position functions go upward
```

```

table_pos_diffs <- t(diff(t(as.matrix(table_to_use))))
ncol_pos_diff_matrix <- ncol(table_pos_diffs)
first_col_mean <- mean(as.numeric(unlist(table_pos_diffs[,1])))
potential_u_shape <- 0
if(first_col_mean < 0){ # downward, so potential U-shape
  # take only negative values from the table
  filtered_table_pos_diffs<-table_pos_diffs
  filtered_table_pos_diffs[table_pos_diffs >= 0] <- NA
  potential_u_shape <- 1
}else{
  # upward - use the positive values
  # take only negative values from the table
  filtered_table_pos_diffs<-table_pos_diffs
  filtered_table_pos_diffs[table_pos_diffs <= 0] <- NA
}
average_pos_diffs <- apply(filtered_table_pos_diffs,2,mean,na.rm=TRUE)
OA_mean_pos_diff <- mean(average_pos_diffs,na.rm=TRUE)

table_len_diffs <- diff(as.matrix(table_to_use))
average_len_diffs <- apply(table_len_diffs,1,mean,na.rm=TRUE)
OA_mean_len_diff <- mean(average_len_diffs,na.rm=TRUE)
CurrentLabel<-"mean change in probability for each additional length: "
print(CurrentLabel)

## [1] "mean change in probability for each additional length: "
print(OA_mean_len_diff)

## [1] -0.01340838
results_report_DF <- AddReportLine(results_report_DF, CurrentLabel,OA_mean_len_diff)

CurrentLabel<-"mean change in probability for each additional position (excluding U-shape): "
print(CurrentLabel)

## [1] "mean change in probability for each additional position (excluding U-shape): "
print(OA_mean_pos_diff)

## [1] -0.02124597
results_report_DF <- AddReportLine(results_report_DF, CurrentLabel,OA_mean_pos_diff)

if(potential_u_shape){ # downward, so potential U-shape
  # take only positive values from the table
  filtered_pos_upward_u<-table_pos_diffs
  filtered_pos_upward_u[table_pos_diffs <= 0] <- NA
  average_pos_u_diffs <- apply(filtered_pos_upward_u,
    2,mean,na.rm=TRUE)
  OA_mean_pos_u_diff <- mean(average_pos_u_diffs,na.rm=TRUE)
  if(is.nan(OA_mean_pos_u_diff) | (OA_mean_pos_u_diff <= 0)){
    print("No U-shape in this participant")
    results_report_DF <- AddReportLine(results_report_DF, "U-shape", 0)
    potential_u_shape <- FALSE
  }else{
    results_report_DF <- AddReportLine(results_report_DF, "U-shape", 1)
  }
}

```

```

CurrentLabel<-"Average upward change after U minimum"
print(CurrentLabel)
print(OA_mean_pos_u_diff)
results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, OA_mean_pos_u_diff)

CurrentLabel<-"Proportion of average downward change"
prop_ave_down <- abs(OA_mean_pos_u_diff/OA_mean_pos_diff)
results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, prop_ave_down)
}
}else{
  print("No U-shape in this participant")
  results_report_DF <- AddReportLine(results_report_DF, "U-shape", 0)
}

## [1] "No U-shape in this participant"
if(potential_u_shape){
  n_rows<-nrow(table_to_use)
  downward_dist <- numeric(n_rows)
  upward_dist <- numeric(n_rows)
  for(i in seq(1,n_rows)){
    current_row <- as.numeric(unlist(table_to_use[i,1:9]))
    current_row <- current_row[!is.na(current_row)]
    current_row_len <- length(current_row)
    row_min <- min(current_row,na.rm=TRUE)
    min_pos <- which(current_row == row_min)
    left_max <- max(current_row[1:min_pos],na.rm=TRUE)
    right_max <- max(current_row[min_pos:current_row_len])
    left_diff <- left_max - row_min
    right_diff <- right_max - row_min
    downward_dist[i]<-left_diff
    upward_dist[i]<-right_diff
  }
  print("differences from left max to min for each row: ")
  print(downward_dist)
  print("differences from min to right max for each row: ")
  print(upward_dist)

  # Use the max return upward (min of upward_dist) and then the
  # downward distance from the left for the same row

  print(" ")
  biggest_return_upward_row <- which(upward_dist == max(upward_dist))
  CurrentLabel <- "row with biggest return upward"
  print(CurrentLabel)
  print(biggest_return_upward_row)
  results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, biggest_return_upward_row)

  print(" ")
  CurrentLabel<-"downward distance for row with the largest upward value"
  print(CurrentLabel)
  print(downward_dist[biggest_return_upward_row])
  results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, downward_dist[biggest_return_upward_row])
}

```

```

CurrentLabel<-"return upward value"
print(upward_dist[biggest_return_upward_row])
results_report_DF <- AddReportLine(results_report_DF,
                                   CurrentLabel,
                                   upward_dist[biggest_return_upward_row])

print(" ")
# percentage return upward
percentage_return_upward <- upward_dist[biggest_return_upward_row]/downward_dist[biggest_return_upward_row]
CurrentLabel <- "Return upward as a proportion of the downward distance:"
print(CurrentLabel)
print(percentange_return_upward)
results_report_DF <- AddReportLine(results_report_DF, CurrentLabel,
                                   percentange_return_upward)
}else{
  print("no U-shape in this participant")
}

## [1] "no U-shape in this participant"

FLPModelEquations<-c(
  # models with log frequency, but no three-way interactions (due to difficulty interpreting and small sample sizes)
  "preserved ~ stimlen*log_freq",
  "preserved ~ stimlen+log_freq",
  "preserved ~ pos*log_freq",
  "preserved ~ pos+log_freq",
  "preserved ~ stimlen*log_freq + pos*log_freq",
  "preserved ~ stimlen*log_freq + pos",
  "preserved ~ stimlen + pos*log_freq",
  "preserved ~ stimlen + pos + log_freq",
  "preserved ~ (I(pos^2)+pos)*log_freq",
  "preserved ~ stimlen*log_freq + (I(pos^2) + pos)*log_freq",
  "preserved ~ stimlen*log_freq + I(pos^2) + pos",
  "preserved ~ stimlen + (I(pos^2) + pos)*log_freq",
  "preserved ~ stimlen + I(pos^2) + pos + log_freq",

  # models without frequency
  "preserved ~ 1",
  "preserved ~ stimlen",
  "preserved ~ pos",
  "preserved ~ stimlen + pos",
  "preserved ~ stimlen*pos",
  "preserved ~ I(pos^2)+pos",
  "preserved ~ stimlen + I(pos^2) + pos",
  "preserved ~ stimlen * (I(pos^2) + pos)"
)

FLPRes<-TestModels(FLPModelEquations,PosDat)

## *****
## model index: 7
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##          data = PosDat)
##

```

```

## Coefficients:
## (Intercept)      stimlen      pos      log_freq pos:log_freq
##      2.84880     -0.09964     -0.12635     -0.04252      0.01937
##
## Degrees of Freedom: 3200 Total (i.e. Null);  3196 Residual
## Null Deviance:      2902
## Residual Deviance: 2832 AIC: 2842
## log likelihood: -1415.789
## Nagelkerke R2:  0.03656854
## % pres/err predicted correctly: -874.873
## % of predictable range [ (model-null)/(1-null) ]:  0.0229878
## *****
## model index:  8
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      pos      log_freq
##      2.82544     -0.09474     -0.13164      0.03827
##
## Degrees of Freedom: 3200 Total (i.e. Null);  3197 Residual
## Null Deviance:      2902
## Residual Deviance: 2834 AIC: 2842
## log likelihood: -1417.11
## Nagelkerke R2:  0.03521338
## % pres/err predicted correctly: -875.8537
## % of predictable range [ (model-null)/(1-null) ]:  0.02189384
## *****
## model index:  17
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      pos
##      2.8920     -0.1040     -0.1313
##
## Degrees of Freedom: 3200 Total (i.e. Null);  3198 Residual
## Null Deviance:      2902
## Residual Deviance: 2836 AIC: 2842
## log likelihood: -1418.173
## Nagelkerke R2:  0.03412193
## % pres/err predicted correctly: -876.5652
## % of predictable range [ (model-null)/(1-null) ]:  0.02110016
## *****
## model index:  13
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      I(pos^2)      pos      log_freq
##      3.07311     -0.10321      0.01298     -0.24386      0.03753

```

```

##
## Degrees of Freedom: 3200 Total (i.e. Null); 3196 Residual
## Null Deviance: 2902
## Residual Deviance: 2833 AIC: 2843
## log likelihood: -1416.297
## Nagelkerke R2: 0.03604709
## % pres/err predicted correctly: -875.6795
## % of predictable range [ (model-null)/(1-null) ]: 0.02208817
## *****
## model index: 20
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen I(pos^2) pos
## 3.14443 -0.11254 0.01329 -0.24621
##
## Degrees of Freedom: 3200 Total (i.e. Null); 3197 Residual
## Null Deviance: 2902
## Residual Deviance: 2835 AIC: 2843
## log likelihood: -1417.32
## Nagelkerke R2: 0.03499856
## % pres/err predicted correctly: -876.375
## % of predictable range [ (model-null)/(1-null) ]: 0.02131229
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen log_freq pos stimlen:log_freq log_fr
## 2.82362 -0.09780 0.05381 -0.12540 -0.01486 0
##
## Degrees of Freedom: 3200 Total (i.e. Null); 3195 Residual
## Null Deviance: 2902
## Residual Deviance: 2831 AIC: 2843
## log likelihood: -1415.434
## Nagelkerke R2: 0.03693221
## % pres/err predicted correctly: -874.6745
## % of predictable range [ (model-null)/(1-null) ]: 0.02320919
## *****
## model index: 18
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen pos stimlen:pos
## 3.33583 -0.15846 -0.26601 0.01599
##
## Degrees of Freedom: 3200 Total (i.e. Null); 3197 Residual
## Null Deviance: 2902

```

```

## Residual Deviance: 2835 AIC: 2843
## log likelihood: -1417.635
## Nagelkerke R2: 0.0346748
## % pres/err predicted correctly: -876.4329
## % of predictable range [ (model-null)/(1-null) ]: 0.02124772
## *****
## model index: 12
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen I(pos^2) pos log_freq I(pos
## 3.126934 -0.109208 0.014378 -0.251046 -0.083450
##
## Degrees of Freedom: 3200 Total (i.e. Null); 3194 Residual
## Null Deviance: 2902
## Residual Deviance: 2829 AIC: 2843
## log likelihood: -1414.681
## Nagelkerke R2: 0.03770435
## % pres/err predicted correctly: -874.7222
## % of predictable range [ (model-null)/(1-null) ]: 0.02315603
## *****
## model index: 6
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen log_freq pos stimlen:log_freq
## 2.82271 -0.09450 0.04979 -0.13163 -0.00148
##
## Degrees of Freedom: 3200 Total (i.e. Null); 3196 Residual
## Null Deviance: 2902
## Residual Deviance: 2834 AIC: 2844
## log likelihood: -1417.106
## Nagelkerke R2: 0.03521776
## % pres/err predicted correctly: -875.8591
## % of predictable range [ (model-null)/(1-null) ]: 0.02188783
## *****
## model index: 11
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen log_freq I(pos^2) pos stimlen:log
## 3.0712546 -0.1030664 0.0440262 0.0129602 -0.2437036 -0.0
##
## Degrees of Freedom: 3200 Total (i.e. Null); 3195 Residual
## Null Deviance: 2902
## Residual Deviance: 2833 AIC: 2845
## log likelihood: -1416.296
## Nagelkerke R2: 0.03604847

```

```

## % pres/err predicted correctly: -875.6808
## % of predictable range [ (model-null)/(1-null) ]: 0.02208674
## *****
## model index: 10
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)          stimlen          log_freq          I(pos^2)          pos          stimlen:I(pos^2)
## 3.103319        -0.107489        0.018060        0.014465        -0.250610
## log_freq:pos
## 0.037848
##
## Degrees of Freedom: 3200 Total (i.e. Null); 3193 Residual
## Null Deviance: 2902
## Residual Deviance: 2829 AIC: 2845
## log likelihood: -1414.376
## Nagelkerke R2: 0.03801664
## % pres/err predicted correctly: -874.4894
## % of predictable range [ (model-null)/(1-null) ]: 0.02341563
## *****
## model index: 21
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)          stimlen          I(pos^2)          pos          stimlen:I(pos^2)          stimlen:I(pos^2):pos
## 3.243388        -0.128748        0.009200        -0.262066        0.000197
##
## Degrees of Freedom: 3200 Total (i.e. Null); 3195 Residual
## Null Deviance: 2902
## Residual Deviance: 2835 AIC: 2847
## log likelihood: -1417.275
## Nagelkerke R2: 0.0350441
## % pres/err predicted correctly: -876.3618
## % of predictable range [ (model-null)/(1-null) ]: 0.02132702
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)          pos          log_freq
## 2.22994        -0.16210        0.05316
##
## Degrees of Freedom: 3200 Total (i.e. Null); 3198 Residual
## Null Deviance: 2902
## Residual Deviance: 2842 AIC: 2848
## log likelihood: -1421.236
## Nagelkerke R2: 0.03097483
## % pres/err predicted correctly: -878.0597

```

```

## % of predictable range [ (model-null)/(1-null) ]: 0.01943314
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) pos log_freq pos:log_freq
## 2.22301 -0.15883 -0.01377 0.01618
##
## Degrees of Freedom: 3200 Total (i.e. Null); 3197 Residual
## Null Deviance: 2902
## Residual Deviance: 2841 AIC: 2849
## log likelihood: -1420.319
## Nagelkerke R2: 0.0319175
## % pres/err predicted correctly: -877.3489
## % of predictable range [ (model-null)/(1-null) ]: 0.02022601
## *****
## model index: 16
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) pos
## 2.2428 -0.1662
##
## Degrees of Freedom: 3200 Total (i.e. Null); 3199 Residual
## Null Deviance: 2902
## Residual Deviance: 2847 AIC: 2851
## log likelihood: -1423.37
## Nagelkerke R2: 0.02877735
## % pres/err predicted correctly: -879.4734
## % of predictable range [ (model-null)/(1-null) ]: 0.0178562
## *****
## model index: 9
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) I(pos^2) pos log_freq I(pos^2):log_freq
## 2.339399 0.007537 -0.226361 -0.056058 -0.002570
##
## Degrees of Freedom: 3200 Total (i.e. Null); 3195 Residual
## Null Deviance: 2902
## Residual Deviance: 2840 AIC: 2852
## log likelihood: -1419.892
## Nagelkerke R2: 0.03235626
## % pres/err predicted correctly: -877.4256
## % of predictable range [ (model-null)/(1-null) ]: 0.02014042
## *****
## model index: 19

```

```

##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)          pos
## 2.339295      0.006442      -0.223211
##
## Degrees of Freedom: 3200 Total (i.e. Null); 3198 Residual
## Null Deviance: 2902
## Residual Deviance: 2846 AIC: 2852
## log likelihood: -1423.162
## Nagelkerke R2: 0.02899198
## % pres/err predicted correctly: -879.5106
## % of predictable range [ (model-null)/(1-null) ]: 0.01781467
## *****
## model index: 15
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
## 2.9018      -0.1728
##
## Degrees of Freedom: 3200 Total (i.e. Null); 3199 Residual
## Null Deviance: 2902
## Residual Deviance: 2866 AIC: 2870
## log likelihood: -1432.973
## Nagelkerke R2: 0.01885498
## % pres/err predicted correctly: -885.1918
## % of predictable range [ (model-null)/(1-null) ]: 0.01147747
## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      log_freq
## 2.83791      -0.16407      0.03683
##
## Degrees of Freedom: 3200 Total (i.e. Null); 3198 Residual
## Null Deviance: 2902
## Residual Deviance: 2864 AIC: 2870
## log likelihood: -1431.979
## Nagelkerke R2: 0.01988535
## % pres/err predicted correctly: -884.6713
## % of predictable range [ (model-null)/(1-null) ]: 0.01205801
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)

```

```
BestFLPModel<-FLPres$ModelResult[[1]]
BestFLPModelFormula<-FLPres$Model[[1]]

FLPAICSummary<-data.frame(Model=FLPres$Model,
                           AIC=FLPres$AIC,row.names=FLPres$Model)
FLPAICSummary$DeltaAIC<-FLPAICSummary$AIC-FLPAICSummary$AIC[1]
FLPAICSummary$AICexp<-exp(-0.5*FLPAICSummary$DeltaAIC)
FLPAICSummary$AICwt<-FLPAICSummary$AICexp/sum(FLPAICSummary$AICexp)
FLPAICSummary$NagR2<-FLPres$NagR2

FLPAICSummary <- merge(FLPAICSummary,FLPres$CoefficientValues,
                      by='row.names',sort=FALSE)
FLPAICSummary <- subset(FLPAICSummary, select = -c(Row.names))

write.csv(FLPAICSummary,paste0(TablesDir,CurPat,"_",
                              RunLabel,"_",CurTask,
                              "_freq_len_pos_model_summary.csv"),row.names = FALSE)

kable(FLPAICSummary)
```

27

Model	AIC Delta	AICc	AICw	NagR ²	(Intercept)	log_freq	stimlen	phg_freq	log_freq	I(pos^2)	pos^2	log_freq	I(pos^2)	stimlen:I(pos^2)
preserved ~ stimlen + pos + log_freq	2842.274	2842.002	2842.772	0.655	52.835	0.0382721	-	NA	NA	NA	NA	NA	NA	NA
						0.0947361		0.1316376						
preserved ~ stimlen + pos	2842.376	2842.067	2842.811	0.699	52.891	0.0999	NA	NA	-	NA	NA	NA	NA	NA
						0.1040485		0.1312586						
preserved ~ stimlen + I(pos^2) + pos + log_freq	2842.501	2842.101	2842.923	0.536	52.473	0.0375235	-	NA	NA	0.0129770	NA	NA	NA	NA
						0.1032073		0.2438555						
preserved ~ stimlen + I(pos^2) + pos	2842.636	2842.108	2842.840	0.023	49.861	0.432	NA	NA	-	NA	NA	0.0132897	NA	NA
						0.1125385		0.2462129						
preserved ~ stimlen * log_freq + pos *	2842.829	2842.058	2842.192	0.453	52.323	0.619	0.0538094	-	NA	0.0240037	NA	NA	NA	NA
						0.0978027	0.0148611	0.253965						
preserved ~ stimlen * pos	2843.269	2842.107	2842.373	0.373	48.345	0.830	NA	NA	-	NA	NA	NA	NA	0.0159942
						0.1584589		0.2660080						
preserved ~ stimlen + (I(pos^2) + pos) * log_freq	2843.362	2843.001	2843.169	0.487	52.469	0.34	-	NA	-	0.0398715	0.0143782	NA	NA	NA
						0.1092083	0.2510456				0.0020850			
preserved ~ stimlen * log_freq + pos	2844.262	2843.386	2843.955	0.335	52.872	0.709	0.0497928	-	NA	NA	NA	NA	NA	NA
						0.0945043	0.0014798	0.16273						
preserved ~ stimlen * log_freq + I(pos^2) + pos	2844.392	2844.182	2844.637	0.338	52.487	0.255	0.0440262	-	NA	NA	0.0129602	NA	NA	NA
						0.1030664	0.0008342	0.2437036						
preserved ~ stimlen * (I(pos^2) + pos) + log_freq	2844.737	2843.388	2844.085	0.538	52.163	0.319	0.0180603	-	NA	0.0378160	0.4648	-	NA	NA
						0.1074887	0.0139832	0.2506105				0.0013661		
preserved ~ stimlen * (I(pos^2) + pos)	2846.457	2844.130	2844.280	0.035	52.443	0.388	NA	NA	-	NA	NA	0.0091995	NA	0.0043390
						0.1287476		0.2620656						0.197
preserved ~ pos + log_freq	2848.679	2848.073	2848.555	0.702	22.991	NA	0.0531612	-	NA	NA	NA	NA	NA	NA
								0.1621000						
preserved ~ pos *	2848.735	2848.902	2849.006	0.335	12.230	NA	-	NA	-	0.0161827	NA	NA	NA	NA
						0.0137662		0.1588317						

Model	AIC	Delta AIC	C _w	Nag R ²	Intercept	stimlen	log_freq	pos	log_freq	pos:log_freq	pos^2	log_freq:pos	len:I(pos^2)
preserved ~ pos	2850.746	2083.004	0.027	0.028	2.84880	NA	NA	-	NA	NA	NA	NA	NA
preserved ~ (I(pos^2) + pos) * log_freq	2851.782	2063.040	0.027	0.045	2.84880	-	NA	-	0.0398787	0.0075369	NA	NA	NA
						0.0560583	0.2263612			0.0025702			
preserved ~ I(pos^2) + pos	2852.327	2055.074	0.027	0.073	2.84880	NA	NA	-	NA	NA	0.006425	NA	NA
								0.2232112					
preserved ~ stimlen	2869.287	2068.133	0.007	0.001	2.84880	0.1728231	NA	NA	NA	NA	NA	NA	NA
preserved ~ stimlen + log_freq	2869.287	2068.133	0.007	0.001	2.84880	0.1728231	0.0368285	NA	NA	NA	NA	NA	NA
							0.1640666						
preserved ~ stimlen * log_freq	2871.306	2068.133	0.003	0.001	2.84880	0.1637955	0.0499106	NA	NA	NA	NA	NA	NA
							0.0016781						
preserved ~ 1	2904.625	2050.893	0.000	0.000	2.84880	NA	NA	NA	NA	NA	NA	NA	NA

```
print(BestFLPModelFormula)
```

```
## [1] "preserved ~ stimlen + pos * log_freq"
```

```
print(BestFLPModel)
```

```
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos      log_freq pos:log_freq
##      2.84880     -0.09964     -0.12635     -0.04252       0.01937
##
## Degrees of Freedom: 3200 Total (i.e. Null);  3196 Residual
## Null Deviance:      2902
## Residual Deviance: 2832  AIC: 2842
```

```
# do a median split on frequency to plot hf/lf effects (analysis is continuous)
median_freq <- median(PosDat$log_freq)
PosDat$freq_bin[PosDat$log_freq >= median_freq] <- "hf"
PosDat$freq_bin[PosDat$log_freq < median_freq] <- "lf"
```

```
PosDat$FLPFitted <- fitted(BestFLPModel)
```

```
HFDat <- PosDat[PosDat$freq_bin == "hf",]
LFDat <- PosDat[PosDat$freq_bin == "lf",]
```

```
HF_Plot <- plot_len_pos_obs_predicted(HFDat, paste0(CurPat, " - ", RunLabel, " - High frequency"), "FLPFitted")
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## Scale for y is already present. Adding another scale for y, which will replace the existing scale.
```

```

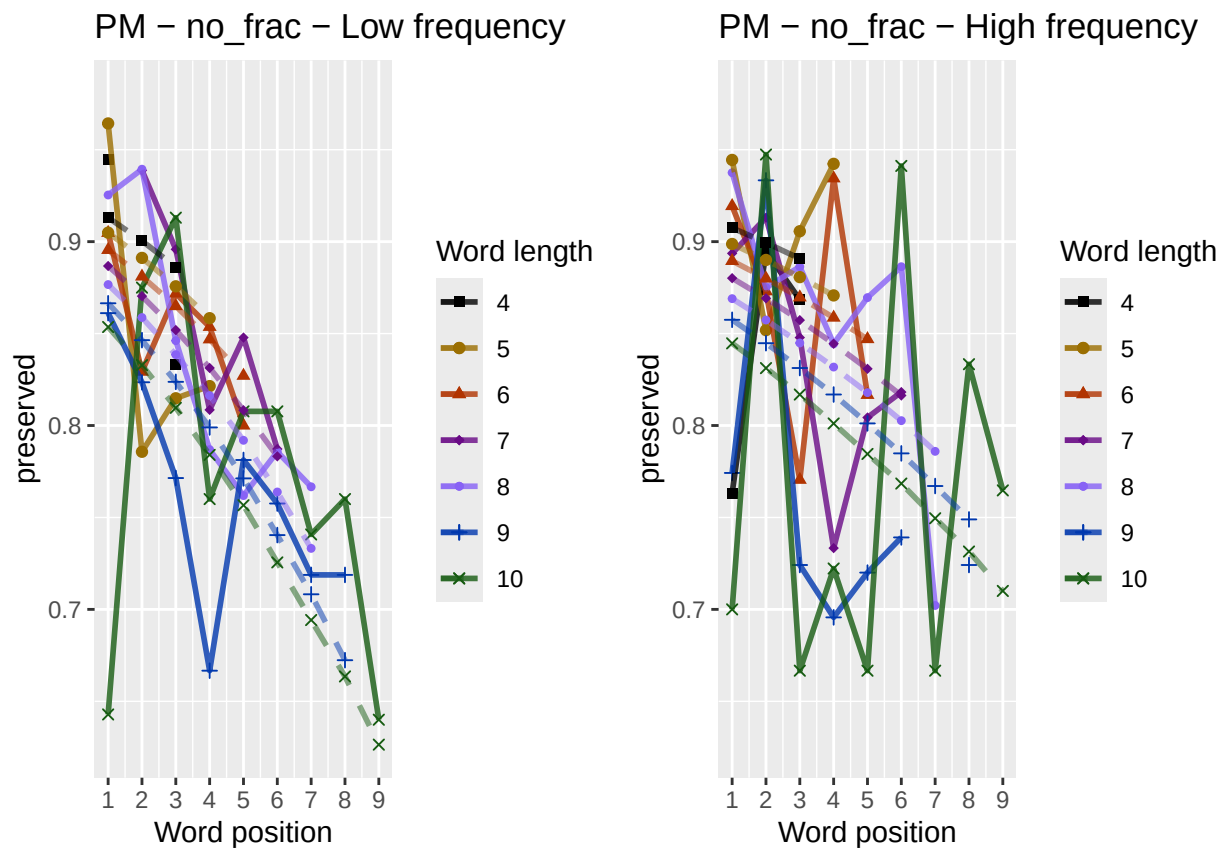
LF_Plot <- plot_len_pos_obs_predicted(LFdat,paste0(CurPat," - ",RunLabel," - Low frequency"),"FLPFitted")

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## Scale for y is already present. Adding another scale for y, which will replace the existing scale.

library(ggpubr)
Both_Plots <- ggarrange(LF_Plot,HF_Plot) # labels=c("LF","HF",ncol=2)

## Warning: Removed 2 rows containing missing values or values outside the scale range (`geom_point()`).
## Warning: Removed 1 row containing missing values or values outside the scale range (`geom_line()`).
## Warning: Removed 1 row containing missing values or values outside the scale range (`geom_point()`).
ggsave(paste0(FigDir,CurPat,"_",RunLabel,"_",
              CurTask,"_frequency_effect_length_pos_wfit.png"),
       device="png",unit="cm",width=30,height=11)
print(Both_Plots)

```



```

# only main effects
MEModelEquations<-c(
  "preserved ~ CumPres",
  "preserved ~ CumErr",
  "preserved ~ (I(pos^2)+pos)",
  "preserved ~ pos",
  "preserved ~ stimlen",
  "preserved ~ 1"
)

```

```

)
MERes<-TestModels(MEModelEquations,PosDat)

## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      1.832      -0.507
##
## Degrees of Freedom: 3200 Total (i.e. Null); 3199 Residual
## Null Deviance:      2902
## Residual Deviance: 2820 AIC: 2824
## log likelihood: -1410.114
## Nagelkerke R2: 0.04237641
## % pres/err predicted correctly: -869.1616
## % of predictable range [ (model-null)/(1-null) ]: 0.02935871
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      pos
##      2.2428      -0.1662
##
## Degrees of Freedom: 3200 Total (i.e. Null); 3199 Residual
## Null Deviance:      2902
## Residual Deviance: 2847 AIC: 2851
## log likelihood: -1423.37
## Nagelkerke R2: 0.02877735
## % pres/err predicted correctly: -879.4734
## % of predictable range [ (model-null)/(1-null) ]: 0.0178562
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)      pos
##      2.339295      0.006442      -0.223211
##
## Degrees of Freedom: 3200 Total (i.e. Null); 3198 Residual
## Null Deviance:      2902
## Residual Deviance: 2846 AIC: 2852
## log likelihood: -1423.162
## Nagelkerke R2: 0.02899198
## % pres/err predicted correctly: -879.5106
## % of predictable range [ (model-null)/(1-null) ]: 0.01781467

```

```

## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      2.9018      -0.1728
##
## Degrees of Freedom: 3200 Total (i.e. Null); 3199 Residual
## Null Deviance:      2902
## Residual Deviance: 2866 AIC: 2870
## log likelihood: -1432.973
## Nagelkerke R2: 0.01885498
## % pres/err predicted correctly: -885.1918
## % of predictable range [ (model-null)/(1-null) ]: 0.01147747
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##      1.72432      -0.05669
##
## Degrees of Freedom: 3200 Total (i.e. Null); 3199 Residual
## Null Deviance:      2902
## Residual Deviance: 2897 AIC: 2901
## log likelihood: -1448.731
## Nagelkerke R2: 0.002443962
## % pres/err predicted correctly: -894.2131
## % of predictable range [ (model-null)/(1-null) ]: 0.001414446
## *****
## model index: 6
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)
##      1.597
##
## Degrees of Freedom: 3200 Total (i.e. Null); 3200 Residual
## Null Deviance:      2902
## Residual Deviance: 2902 AIC: 2904
## log likelihood: -1451.064
## Nagelkerke R2: 0
## % pres/err predicted correctly: -895.4811
## % of predictable range [ (model-null)/(1-null) ]: 0
## *****

```

```

BestMEModel<-MERes$ModelResult[[ BestModelIndexL1 ]]
BestMEModelFormula<-MERes$Model[[BestModelIndexL1]]

MEAICSummary<-data.frame(Model=MERes$Model,
                          AIC=MERes$AIC,row.names=MERes$Model)
MEAICSummary$DeltaAIC<-MEAICSummary$AIC-MEAICSummary$AIC[1]
MEAICSummary$AICexp<-exp(-0.5*MEAICSummary$DeltaAIC)
MEAICSummary$AICwt<-MEAICSummary$AICexp/sum(MEAICSummary$AICexp)
MEAICSummary$NagR2<-MERes$NagR2

MEAICSummary <- merge(MEAICSummary,MERes$CoefficientValues,
                      by='row.names',sort=FALSE)
MEAICSummary <- subset(MEAICSummary, select = -c(Row.names))

write.csv(MEAICSummary,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                             CurTask,"_main_effects_model_summary.csv"),
          row.names = FALSE)
kable(MEAICSummary)

```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(pos^2)	pos	stimlen
preserved ~ CumErr	2824.228	0.00000	1.0e+00	0.9999975	0.0423761	1.832194	NA	-	NA	NA	NA
								0.5070111			
preserved ~ pos	2850.742	26.51179	1.8e-06	0.0000017	0.0287772	2.242829	NA	NA	NA	-	NA
										0.1661569	
preserved ~ (I(pos^2) + pos)	2852.322	28.09503	3.0e-07	0.0000008	0.0289922	2.339295	NA	NA	0.0064425	-	NA
										0.2232112	
preserved ~ stimlen	2869.947	45.71812	2.0e+00	0.0000000	0.0188552	2.901848	NA	NA	NA	NA	-
											0.1728231
preserved ~ CumPres	2901.467	77.23340	0.0e+00	0.0000000	0.0024440	0.724318	-	NA	NA	NA	NA
								0.0566946			
preserved ~ 1	2904.127	99.90040	0.0e+00	0.0000000	0.0000000	0.597117	NA	NA	NA	NA	NA

```

if(DoSimulations){
  BestMEModelFormulaRnd <- BestMEModelFormula
  if(grepl("CumPres",BestMEModelFormulaRnd)){
    BestMEModelFormulaRnd <- gsub("CumPres","RndCumPres",BestMEModelFormulaRnd)
  }else if(grepl("CumErr",BestMEModelFormulaRnd)){
    BestMEModelFormulaRnd <- gsub("CumErr","RndCumErr",BestMEModelFormulaRnd)
  }

  RndModelAIC<-numeric(length=RandomSamples)
  for(rindex in seq(1,RandomSamples)){
    # Shuffle cumulative values
    PosDat$RndCumPres <- ShuffleWithinWord(PosDat,"CumPres")
    PosDat$RndCumErr <- ShuffleWithinWord(PosDat,"CumErr")
    BestModelRnd <- glm(as.formula(BestMEModelFormulaRnd),
                      family="binomial",data=PosDat)
    RndModelAIC[rindex] <- BestModelRnd$aic
  }
  ModelNames<-c(paste0("***",BestMEModelFormula),
                rep(BestMEModelFormulaRnd,RandomSamples))
}

```

```

AICValues <- c(BestMEModel$aic,RndModelAIC)
BestMEModelRndDF <- data.frame(Name=ModelNames,AIC=AICValues)
BestMEModelRndDF <- BestMEModelRndDF %>% arrange(AIC)
BestMEModelRndDF <- rbind(BestMEModelRndDF,
                           data.frame(Name=c("Random average"),
                                         AIC=c(mean(RndModelAIC))))
BestMEModelRndDF <- rbind(BestMEModelRndDF,
                           data.frame(Name=c("Random SD"),
                                         AIC=c(sd(RndModelAIC))))

write.csv(BestMEModelRndDF,
          paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                 "_best_main_effects_model_with_random_cum_term.csv"),
          row.names = FALSE)
}

syll_component_summary <- PosDat %>%
  group_by(syll_component) %>% summarise(MeanPres = mean(preserved),
                                         N = n())
write.csv(syll_component_summary,paste0(TablesDir,CurPat,"_",
                                         RunLabel,"_",CurTask,
                                         "_syllable_component_summary.csv"),row.names = FALSE)
kable(syll_component_summary)

```

syll_component	MeanPres	N
1	0.8465347	404
O	0.8263229	1474
P	0.8076923	26
S	0.9050000	200
V	0.8204193	1097

```

# main effects models for data without satellite positions

keep_components = c("0","V","1")
OV1Data <- PosDat[PosDat$syll_component %in% keep_components,]
OV1Data <- OV1Data %>% select(stim_number,
                             stimlen,stim,pos,
                             preserved,syll_component)
OV1Data$CumPres <- CalcCumPres(OV1Data)
OV1Data$CumErr <- CalcCumErrFromPreserved(OV1Data)

SimpSyllMEAICSummary <- EvaluateSubsetData(OV1Data,MEModelEquations)

## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##          data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      1.7798      -0.4965
##

```

```

## Degrees of Freedom: 2974 Total (i.e. Null); 2973 Residual
## Null Deviance: 2742
## Residual Deviance: 2672 AIC: 2676
## log likelihood: -1336.207
## Nagelkerke R2: 0.03820867
## % pres/err predicted correctly: -828.4637
## % of predictable range [ (model-null)/(1-null) ]: 0.02610525
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) pos
## 2.2031 -0.1644
##
## Degrees of Freedom: 2974 Total (i.e. Null); 2973 Residual
## Null Deviance: 2742
## Residual Deviance: 2690 AIC: 2694
## log likelihood: -1345.157
## Nagelkerke R2: 0.02841556
## % pres/err predicted correctly: -835.6598
## % of predictable range [ (model-null)/(1-null) ]: 0.01765615
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) I(pos^2) pos
## 2.35969 0.01049 -0.25700
##
## Degrees of Freedom: 2974 Total (i.e. Null); 2972 Residual
## Null Deviance: 2742
## Residual Deviance: 2689 AIC: 2695
## log likelihood: -1344.636
## Nagelkerke R2: 0.02898675
## % pres/err predicted correctly: -835.6016
## % of predictable range [ (model-null)/(1-null) ]: 0.01772446
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen
## 2.8497 -0.1706
##
## Degrees of Freedom: 2974 Total (i.e. Null); 2973 Residual
## Null Deviance: 2742
## Residual Deviance: 2708 AIC: 2712

```

```
## log likelihood: -1354.029
## Nagelkerke R2: 0.01865053
## % pres/err predicted correctly: -840.944
## % of predictable range [ (model-null)/(1-null) ]: 0.01145185
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) CumPres
## 1.6604 -0.0469
##
## Degrees of Freedom: 2974 Total (i.e. Null); 2973 Residual
## Null Deviance: 2742
## Residual Deviance: 2739 AIC: 2743
## log likelihood: -1369.456
## Nagelkerke R2: 0.001530508
## % pres/err predicted correctly: -849.9372
## % of predictable range [ (model-null)/(1-null) ]: 0.0008926628
## *****
## model index: 6
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)
## 1.564
##
## Degrees of Freedom: 2974 Total (i.e. Null); 2974 Residual
## Null Deviance: 2742
## Residual Deviance: 2742 AIC: 2744
## log likelihood: -1370.827
## Nagelkerke R2: -3.687812e-16
## % pres/err predicted correctly: -850.6975
## % of predictable range [ (model-null)/(1-null) ]: 0
## *****
```

```
write.csv(SimpSyllMEAICSummary,
  paste0(TablesDir, CurPat, "_", RunLabel, "_",
    CurTask,
    "_OV1_main_effects_model_summary.csv"),
  row.names = FALSE)
kable(SimpSyllMEAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(pos^2)	pos	stimlen
preserved ~ CumErr	2676.413	0.00000	1.000000	0.999790	0.038208	7.779769	NA	-	NA	NA	NA
preserved ~ pos	2694.314	17.90110	0.000129	0.000129	0.028415	2.203123	NA	NA	NA	-	NA
								0.4965123		0.1643612	

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(pos^2)	pos	stimlen
preserved ~ (I(pos^2) + pos)	2695.273	18.85996	0.000080	0.000080	0.028986	7.359693	NA	NA	0.0104851	-	NA
										0.2569999	
preserved ~ stimlen	2712.053	35.64426	0.000000	0.000000	0.018650	2.849735	NA	NA	NA	NA	-
											0.1705605
preserved ~ CumPres	2742.916	66.49839	0.000000	0.000000	0.001530	5.660421	-	NA	NA	NA	NA
							0.0469006				
preserved ~ 1	2743.654	67.24119	0.000000	0.000000	0.000000	0.563750	NA	NA	NA	NA	NA

```
# main effects models for data without satellite or coda positions
# (tradeoff -- takes away more complex segments, in addition to satellites, but
# also reduces data)
```

```
keep_components = c("0","V")
OVDData <- PosDat[PosDat$syll_component %in% keep_components,]
OVDData <- OVDData %>% select(stim_number,
                             stimlen,stim,pos,
                             preserved,syll_component)
OVDData$CumPres <- CalcCumPres(OVDData)
OVDData$CumErr <- CalcCumErrFromPreserved(OVDData)

SimpSyllMEAICSummary2<-EvaluateSubsetData(OVDData,MEModelEquations)
```

```
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)          pos
##      2.1960      -0.1692
##
## Degrees of Freedom: 2570 Total (i.e. Null); 2569 Residual
## Null Deviance:      2394
## Residual Deviance: 2344 AIC: 2348
## log likelihood: -1171.869
## Nagelkerke R2: 0.0319467
## % pres/err predicted correctly: -730.4988
## % of predictable range [ (model-null)/(1-null) ]: 0.01992
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)          pos
##      2.39127      0.01348     -0.28759
##
## Degrees of Freedom: 2570 Total (i.e. Null); 2568 Residual
## Null Deviance:      2394
```

```

## Residual Deviance: 2342  AIC: 2348
## log likelihood: -1171.065
## Nagelkerke R2: 0.03295856
## % pres/err predicted correctly: -730.3096
## % of predictable range [ (model-null)/(1-null) ]: 0.02017341
## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      1.7199      -0.4985
##
## Degrees of Freedom: 2570 Total (i.e. Null); 2569 Residual
## Null Deviance: 2394
## Residual Deviance: 2348  AIC: 2352
## log likelihood: -1174.05
## Nagelkerke R2: 0.02919891
## % pres/err predicted correctly: -730.6425
## % of predictable range [ (model-null)/(1-null) ]: 0.01972744
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      2.8230      -0.1703
##
## Degrees of Freedom: 2570 Total (i.e. Null); 2569 Residual
## Null Deviance: 2394
## Residual Deviance: 2364  AIC: 2368
## log likelihood: -1182.11
## Nagelkerke R2: 0.01900193
## % pres/err predicted correctly: -736.5789
## % of predictable range [ (model-null)/(1-null) ]: 0.01177371
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##      1.7539      -0.1189
##
## Degrees of Freedom: 2570 Total (i.e. Null); 2569 Residual
## Null Deviance: 2394
## Residual Deviance: 2382  AIC: 2386
## log likelihood: -1190.98
## Nagelkerke R2: 0.00770572

```

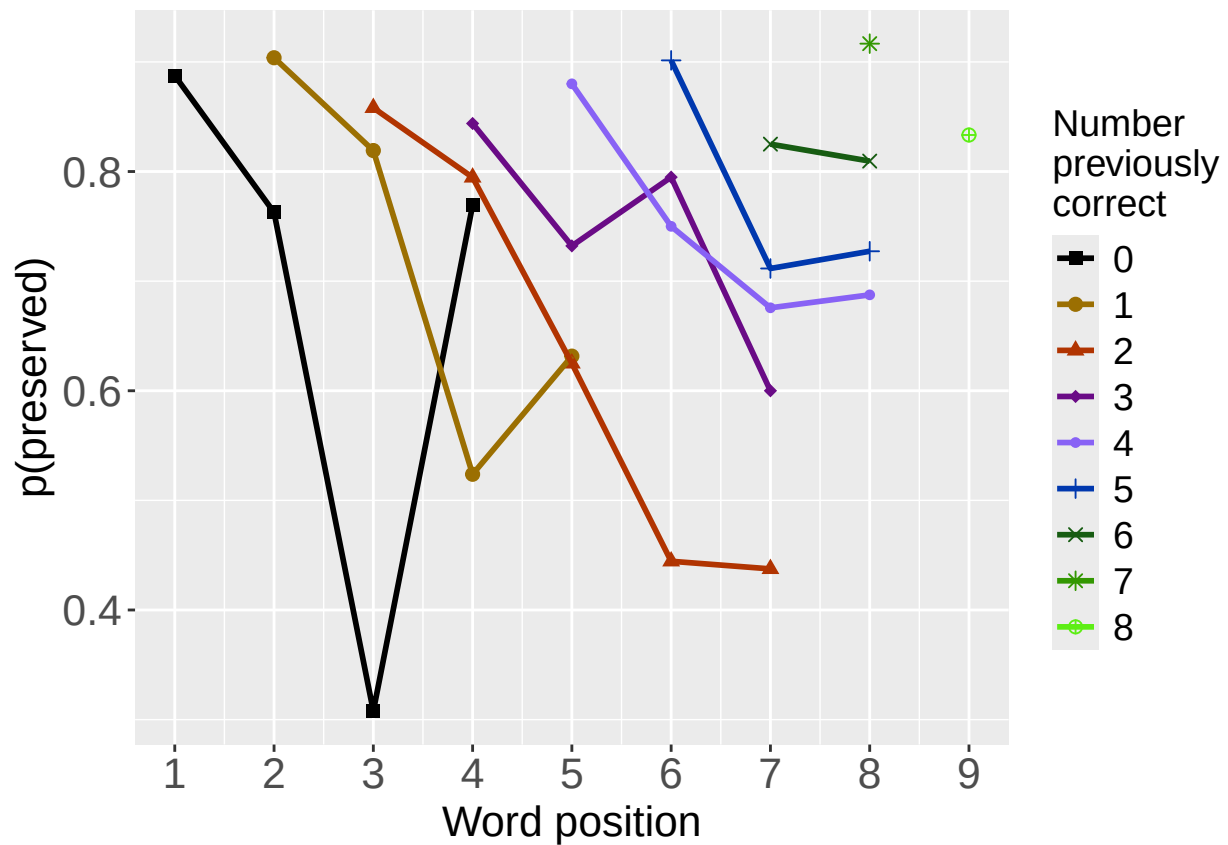
```
## % pres/err predicted correctly: -741.8757
## % of predictable range [ (model-null)/(1-null) ]: 0.004676895
## *****
## model index: 6
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)
## 1.542
##
## Degrees of Freedom: 2570 Total (i.e. Null); 2570 Residual
## Null Deviance: 2394
## Residual Deviance: 2394 AIC: 2396
## log likelihood: -1196.996
## Nagelkerke R2: 0
## % pres/err predicted correctly: -745.3664
## % of predictable range [ (model-null)/(1-null) ]: 0
## *****
```

```
write.csv(SimpSyllMEAICSummary2,
          paste0(TablesDir, CurPat, "_", RunLabel, "_", CurTask,
                "_OV_main_effects_model_summary.csv"), row.names = FALSE)
kable(SimpSyllMEAICSummary2)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(pos^2)	pos	stimlen
preserved ~ pos	2347.730	0.0000000	0.0000000	0.5168066	0.0319467	195991	NA	NA	NA	-	NA
										0.1692106	
preserved ~ (I(pos^2) + pos)	2348.130	0.3921486	0.8219512	0.4247898	0.0329585	391272	NA	NA	0.0134792	-	NA
										0.2875926	
preserved ~ CumErr	2352.104	4.3612150	0.1129728	0.0583851	0.0291989	719897	NA	-	NA	NA	NA
								0.4985009			
preserved ~ stimlen	2368.220	20.4812282	0.0000357	0.0000184	0.0190012	823028	NA	NA	NA	NA	-
											0.1703338
preserved ~ CumPres	2385.963	38.2217323	0.0000000	0.0000000	0.0077057	753931	-	NA	NA	NA	NA
							0.1189189				
preserved ~ 1	2395.994	48.2535843	0.0000000	0.0000000	0.0000000	542335	NA	NA	NA	NA	NA

```
# plot prev err and prev cor plots
PrevCorPlot<-PlotObsPreviousCorrect(PosDat,palette_values,shape_values)
```

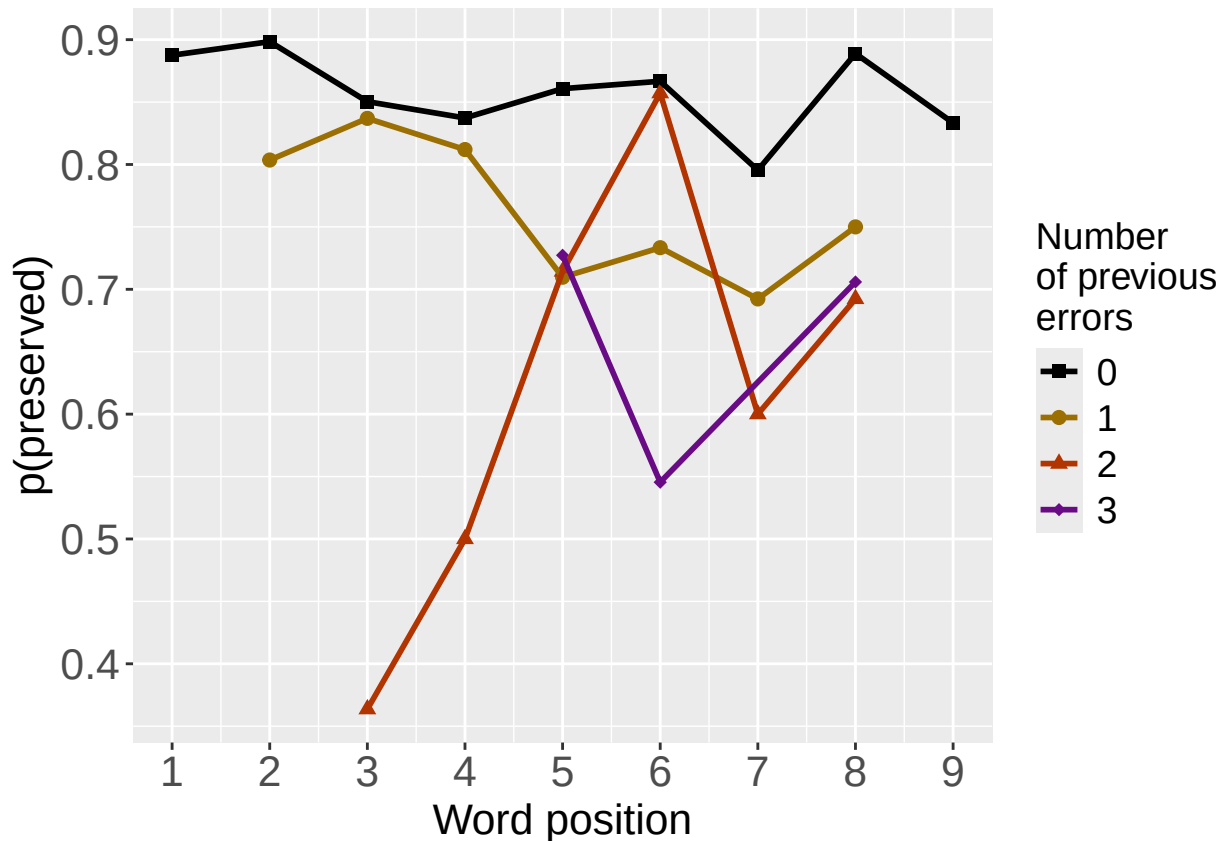
```
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
print(PrevCorPlot)
```



```
PrevErrPlot<-PlotObsPreviousError(PosDat,palette_values,shape_values)
```

```
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
```

```
print(PrevErrPlot)
```



```

PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_PrevCorPlot_Obs")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevCorPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image

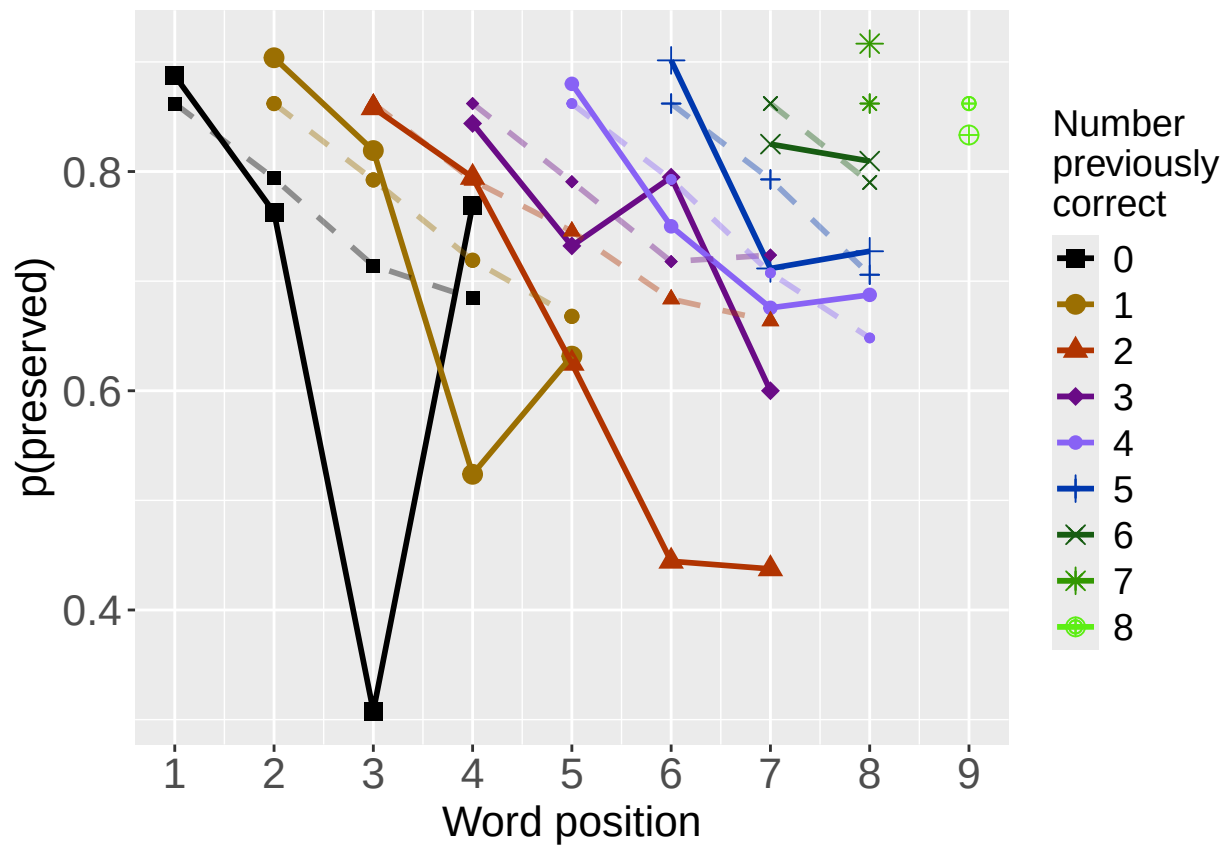
PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_PrevErrPlot_Obs")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevErrPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image
# plot prev err and prev cor with predicted values
MEModel<-MERes$ModelResult[[ BestModelIndexL1 ]]
PosDat$MEPred<-fitted(MEModel)

PrevCorPlot<-PlotObsPredPreviousCorrect(PosDat,"preserved","MEPred",palette_values,shape_values)

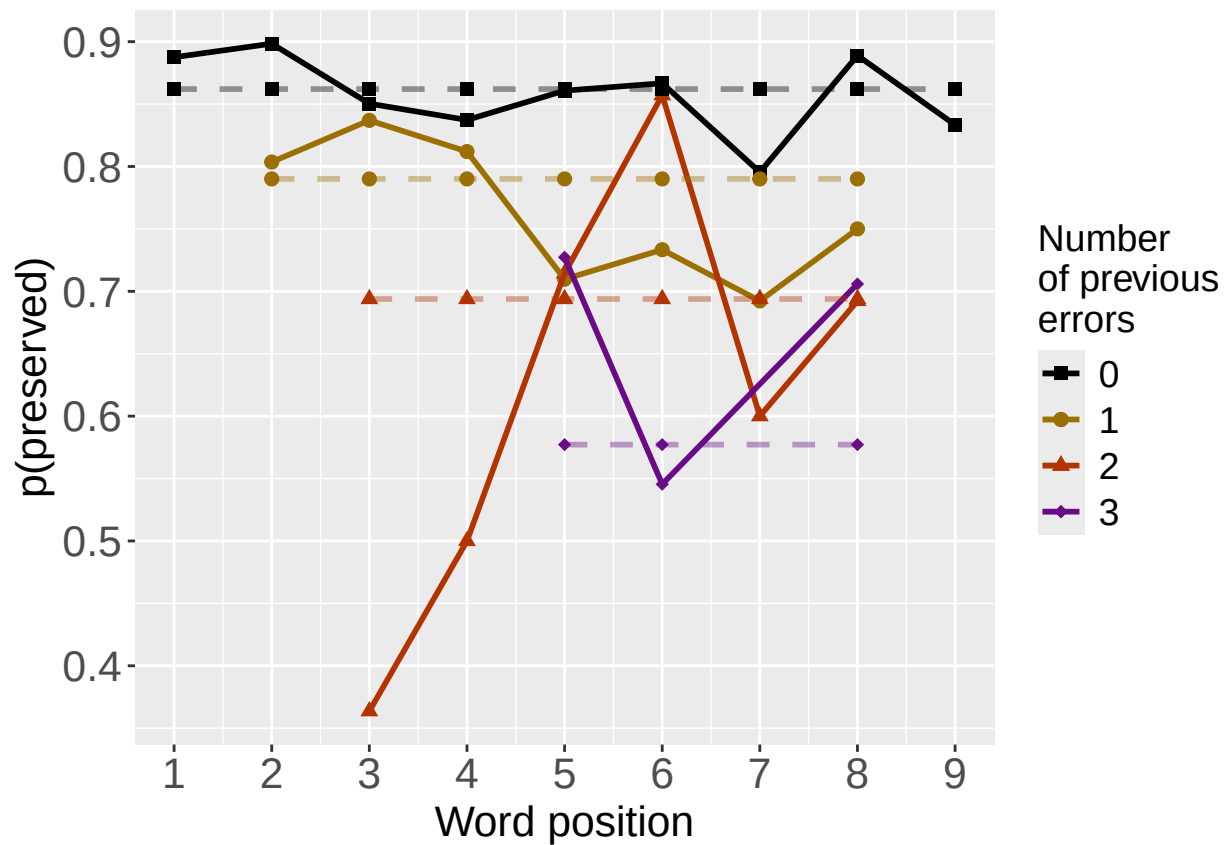
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
print(PrevCorPlot)

```



```
PrevErrPlot<-PlotObsPredPreviousError(PosDat,"preserved","MEPred",palette_values,shape_values)

## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
print(PrevErrPlot)
```



```

PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",
                  CurTask,"PrevCorPlot_ObsPredME")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevCorPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image

PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",
                  CurTask,"PrevErrPlot_ObsPredME")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevErrPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image

```

```

CumAICSummary <- NULL

#####
# level 2 -- Add position squared (quadratic with position)
#####

# After establishing the primary variable, see about additions

BestMEFactor<-BestMEModelFormula
OtherFactor<-"I(pos^2) + pos"
OtherFactorName<-"quadratic_by_pos"

if(!grepl("pos",BestMEFactor,fixed = TRUE)){ # if best model does not include pos
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor,OtherFactorName)
  CumAICSummary <- AICSummary
  kable(AICSummary)
}

```

```

## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      I(pos^2)         pos
##    2.32045    -0.40274     0.01381    -0.20736
##
## Degrees of Freedom: 3200 Total (i.e. Null);  3197 Residual
## Null Deviance:      2902
## Residual Deviance: 2807  AIC: 2815
## log likelihood:  -1403.656
## Nagelkerke R2:  0.04896086
## % pres/err predicted correctly:  -866.003
## % of predictable range [ (model-null)/(1-null) ]:  0.03288198
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",

```

```

##      data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      1.832      -0.507
##
## Degrees of Freedom: 3200 Total (i.e. Null);  3199 Residual
## Null Deviance:      2902
## Residual Deviance: 2820  AIC: 2824
## log likelihood:  -1410.114
## Nagelkerke R2:  0.04237641
## % pres/err predicted correctly:  -869.1616
## % of predictable range [ (model-null)/(1-null) ]:  0.02935871
## *****
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)      pos
##      2.339295      0.006442      -0.223211
##
## Degrees of Freedom: 3200 Total (i.e. Null);  3198 Residual
## Null Deviance:      2902
## Residual Deviance: 2846  AIC: 2852
## log likelihood:  -1423.162
## Nagelkerke R2:  0.02899198
## % pres/err predicted correctly:  -879.5106
## % of predictable range [ (model-null)/(1-null) ]:  0.01781467
## *****

```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	I(pos^2)	pos
preserved ~ CumErr + I(pos^2) + pos	2815.313	0.000000	1.0000000	0.9885466	0.0489609	2.320449	-0.4027384	0.0138076	-0.2073617
preserved ~ CumErr	2824.228	8.915907	0.0115861	0.0114534	0.0423764	1.832194	-0.5070111	NA	NA
preserved ~ I(pos^2) + pos	2852.323	37.010957	0.0000000	0.0000000	0.0289920	2.339295	NA	0.0064425	-0.2232112

```
#####
# level 2 -- Add length
#####

BestMEFactor<-BestMEModelFormula
OtherFactor<-"stimlen"

if(!grepl(OtherFactor,BestMEFactor,fixed = TRUE)){
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor)
  CumAICSummary <- ConcatAndAddMissingColumns(CumAICSummary,AICSummary)
  kable(AICSummary)
}

## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) CumErr stimlen
## 2.6260 -0.4443 -0.1092
##
## Degrees of Freedom: 3200 Total (i.e. Null); 3198 Residual
## Null Deviance: 2902
## Residual Deviance: 2807 AIC: 2813
## log likelihood: -1403.602
## Nagelkerke R2: 0.04901653
## % pres/err predicted correctly: -865.4967
## % of predictable range [ (model-null)/(1-null) ]: 0.03344677
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) CumErr
## 1.832 -0.507
##
## Degrees of Freedom: 3200 Total (i.e. Null); 3199 Residual
## Null Deviance: 2902
## Residual Deviance: 2820 AIC: 2824
## log likelihood: -1410.114
## Nagelkerke R2: 0.04237641
## % pres/err predicted correctly: -869.1616
## % of predictable range [ (model-null)/(1-null) ]: 0.02935871
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
```

```
## (Intercept)      stimlen
##      2.9018      -0.1728
##
## Degrees of Freedom: 3200 Total (i.e. Null);  3199 Residual
## Null Deviance:      2902
## Residual Deviance: 2866  AIC: 2870
## log likelihood:  -1432.973
## Nagelkerke R2:   0.01885498
## % pres/err predicted correctly:  -885.1918
## % of predictable range [ (model-null)/(1-null) ]:  0.01147747
## *****
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	stimlen
preserved ~ CumErr + stimlen	2813.203	0.00000	1.0000000	0.9959809	0.0490165	2.626042	- 0.4442567	- 0.1091501
preserved ~ CumErr	2824.228	11.02532	0.0040354	0.0040191	0.0423764	1.832194	- 0.5070111	NA
preserved ~ stimlen	2869.947	56.74343	0.0000000	0.0000000	0.0188550	2.901848	NA	- 0.1728231

```
#####
# level 2 -- add cumulative preserved
#####

BestMEFactor<-BestMEModelFormula
OtherFactor<-"CumPres"

if(!grepl(OtherFactor,BestMEFactor,fixed = TRUE)){
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor)
  CumAICSummary <- ConcatAndAddMissingColumns(CumAICSummary,AICSummary)
  kable(AICSummary)
}
```

```
## *****
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      1.832      -0.507
##
## Degrees of Freedom: 3200 Total (i.e. Null);  3199 Residual
## Null Deviance:      2902
## Residual Deviance: 2820  AIC: 2824
## log likelihood:  -1410.114
## Nagelkerke R2:   0.04237641
## % pres/err predicted correctly:  -869.1616
## % of predictable range [ (model-null)/(1-null) ]:  0.02935871
## *****
## model index:  2
##
```

```
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      CumPres
##      1.9110      -0.5000      -0.0364
##
## Degrees of Freedom: 3200 Total (i.e. Null);  3198 Residual
## Null Deviance:      2902
## Residual Deviance: 2818  AIC: 2824
## log likelihood:  -1409.214
## Nagelkerke R2:  0.04329565
## % pres/err predicted correctly:  -868.8813
## % of predictable range [ (model-null)/(1-null) ]:  0.02967132
## *****
## model index:  3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##      1.72432      -0.05669
##
## Degrees of Freedom: 3200 Total (i.e. Null);  3199 Residual
## Null Deviance:      2902
## Residual Deviance: 2897  AIC: 2901
## log likelihood:  -1448.731
## Nagelkerke R2:  0.002443962
## % pres/err predicted correctly:  -894.2131
## % of predictable range [ (model-null)/(1-null) ]:  0.001414446
## *****
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	CumPres
preserved ~ CumErr	2824.228	0.0000000	1.0000000	0.5249773	0.0423764	1.832194	-	NA
							0.5070111	
preserved ~ CumErr	2824.428	0.1999849	0.9048443	0.4750227	0.0432956	1.910972	-	-
+ CumPres							0.5000358	0.0363958
preserved ~ CumPres	2901.462	77.2334938	0.0000000	0.0000000	0.0024440	1.724318	NA	-
								0.0566946

```
#####
# level 2 -- Add linear position (NOT quadratic)
#####

BestMEFactor<-BestMEModelFormula
OtherFactor<-"pos"

if(!grepl(OtherFactor,BestMEFactor,fixed = TRUE)){
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor)
  CumAICSummary <- ConcatAndAddMissingColumns(CumAICSummary,AICSummary)
  kable(AICSummary)
}
```

```

## *****
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##         data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      pos
##      2.11907      -0.39327      -0.08757
##
## Degrees of Freedom: 3200 Total (i.e. Null);  3198 Residual
## Null Deviance:      2902
## Residual Deviance: 2809  AIC: 2815
## log likelihood:  -1404.582
## Nagelkerke R2:  0.04801846
## % pres/err predicted correctly:  -866.301
## % of predictable range [ (model-null)/(1-null) ]:  0.0325496
## *****
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##         data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      1.832      -0.507
##
## Degrees of Freedom: 3200 Total (i.e. Null);  3199 Residual
## Null Deviance:      2902
## Residual Deviance: 2820  AIC: 2824
## log likelihood:  -1410.114
## Nagelkerke R2:  0.04237641
## % pres/err predicted correctly:  -869.1616
## % of predictable range [ (model-null)/(1-null) ]:  0.02935871
## *****
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##         data = PosDat)
##
## Coefficients:
## (Intercept)      pos
##      2.2428      -0.1662
##
## Degrees of Freedom: 3200 Total (i.e. Null);  3199 Residual
## Null Deviance:      2902
## Residual Deviance: 2847  AIC: 2851
## log likelihood:  -1423.37
## Nagelkerke R2:  0.02877735
## % pres/err predicted correctly:  -879.4734
## % of predictable range [ (model-null)/(1-null) ]:  0.0178562
## *****

```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	pos
preserved ~ CumErr + pos	2815.164	0.000000	1.000000	0.9893559	0.0480185	2.119070	- 0.3932685	- 0.0875689
preserved ~ CumErr	2824.228	9.064108	0.0107586	0.0106440	0.0423764	1.832194	- 0.5070111	NA
preserved ~ pos	2850.740	35.575894	0.0000000	0.0000000	0.0287773	2.242829	NA	- 0.1661569

```
CumAICSummary <- CumAICSummary %>% arrange(AIC)
BestModelFormulaL2 <- CumAICSummary$Model[BestModelIndexL2]
write.csv(CumAICSummary, paste0(TablesDir, CurPat, "_",
                                RunLabel, "_", CurTask,
                                "_main_effects_plus_one_model_summary.csv"), row.names = FALSE)
kable(CumAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	I(pos^2)	pos	stimlen	CumPres
preserved ~ CumErr + stimlen	2813.208	0.000000	0.000000	0.995980	0.0490165	2.626042	- 0.4442567	NA	NA	- 0.1091501	NA
preserved ~ CumErr + pos	2815.164	0.000000	0.000000	0.9893559	0.0480185	2.119070	- 0.3932685	NA	-	NA	NA
preserved ~ CumErr + I(pos^2) + pos	2815.318	0.000000	0.000000	0.9885466	0.0489602	2.320449	- 0.4027384	0.0138076	-	NA	NA
preserved ~ CumErr	2824.228	9.159066	0.0115861	0.0114534	0.0423764	1.832194	- 0.5070111	NA	NA	NA	NA
preserved ~ CumErr	2824.228	1.025316	0.0040354	0.0040190	0.0423764	1.832194	- 0.5070111	NA	NA	NA	NA
preserved ~ CumErr	2824.228	0.000000	0.000000	0.5249773	0.0423764	1.832194	- 0.5070111	NA	NA	NA	NA
preserved ~ CumErr	2824.228	0.064108	0.0107586	0.0106440	0.0423764	1.832194	- 0.5070111	NA	NA	NA	NA
preserved ~ CumErr + CumPres	2824.428	1.199984	0.0904844	0.0475022	0.0432956	1.910972	- 0.5000358	NA	NA	NA	- 0.0363958
preserved ~ pos	2850.740	35.575894	0.000000	0.000000	0.0287773	2.242829	NA	NA	- 0.1661569	NA	NA
preserved ~ I(pos^2) + pos	2852.323	7.010956	0.000000	0.000000	0.0289920	2.339295	NA	0.0064425	- 0.2232112	NA	NA
preserved ~ stimlen	2869.945	56.743430	0.000000	0.000000	0.0188530	2.901848	NA	NA	NA	- 0.1728231	NA
preserved ~ CumPres	2901.462	7.233493	0.000000	0.000000	0.0024440	2.724318	NA	NA	NA	NA	- 0.0566946

```
# explore influence of frequency and length

if(grepl("stimlen", BestModelFormulaL2)){ # if length is already in the formula
  Level3ModelEquations<-c(
    BestModelFormulaL2, # best model from level 2
    "preserved ~ 1", # null model
    paste0(BestModelFormulaL2, " + log_freq")
  )
}
```

```

}else if(grepl("log_freq",BestModelFormulaL2)){ # if log_freq is already in the formula
  Level3ModelEquations<-c(
    BestModelFormulaL2, # best model from level 2
    "preserved ~ 1", # null model
    paste0(BestModelFormulaL2," + stimlen")
  )
}else{
  Level3ModelEquations<-c(
    BestModelFormulaL2, # best model from level 2
    "preserved ~ 1", # null model
    paste0(BestModelFormulaL2," + log_freq"),
    paste0(BestModelFormulaL2," + stimlen"),
    paste0(BestModelFormulaL2," + stimlen + log_freq")
  )
}
}

Level3Res<-TestModels(Level3ModelEquations,PosDat)

```

```

## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      stimlen      log_freq
##      2.55881      -0.44506      -0.09989      0.03877
##
## Degrees of Freedom: 3200 Total (i.e. Null); 3197 Residual
## Null Deviance:      2902
## Residual Deviance: 2805 AIC: 2813
## log likelihood: -1402.524
## Nagelkerke R2: 0.05011264
## % pres/err predicted correctly: -864.8858
## % of predictable range [ (model-null)/(1-null) ]: 0.03412825
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      stimlen
##      2.6260      -0.4443      -0.1092
##
## Degrees of Freedom: 3200 Total (i.e. Null); 3198 Residual
## Null Deviance:      2902
## Residual Deviance: 2807 AIC: 2813
## log likelihood: -1403.602
## Nagelkerke R2: 0.04901653
## % pres/err predicted correctly: -865.4967
## % of predictable range [ (model-null)/(1-null) ]: 0.03344677
## *****
## model index: 2

```

```
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)
## 1.597
##
## Degrees of Freedom: 3200 Total (i.e. Null); 3200 Residual
## Null Deviance: 2902
## Residual Deviance: 2902 AIC: 2904
## log likelihood: -1451.064
## Nagelkerke R2: 0
## % pres/err predicted correctly: -895.4811
## % of predictable range [ (model-null)/(1-null) ]: 0
## *****
```

```
BestModelL3<-Level3Res$ModelResult[[ BestModelIndexL3 ]]
BestModelFormulaL3<-Level3Res$Model[[BestModelIndexL3]]

AICSummary<-data.frame(Model=Level3Res$Model,
                        AIC=Level3Res$AIC,
                        row.names = Level3Res$Model)
AICSummary$DeltaAIC<-AICSummary$AIC-AICSummary$AIC[1]
AICSummary$AICexp<-exp(-0.5*AICSummary$DeltaAIC)
AICSummary$AICwt<-AICSummary$AICexp/sum(AICSummary$AICexp)
AICSummary$NagR2<-Level3Res$NagR2

AICSummary <- merge(AICSummary,Level3Res$CoefficientValues,
                    by='row.names',sort=FALSE)
AICSummary <- subset(AICSummary, select = -c(Row.names))

write.csv(AICSummary,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                          CurTask,"_main_effects_plus_one_len_freq_summary.csv"),
          row.names = FALSE)

kable(AICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	stimlen	log_freq
preserved ~ CumErr + stimlen + log_freq	2813.0480	0.00000001	0.00000000	0.5193978	0.0501122	0.558809	-	-	0.0387742
preserved ~ CumErr + stimlen	2813.2030	0.15526030	0.9253060	0.4806022	0.0490162	0.626042	-	-	NA
preserved ~ 1	2904.1299	1.0809718	0.0000000	0.0000000	0.0000000	1.597117	NA	NA	NA

```
# BestModelIndex is set by the parameter file and is used to choose
# a model that is essentially equivalent to the lowest AIC model, but
# simpler (and with a slightly higher AIC value, so not in first position)
BestModelL3<-Level3Res$ModelResult[[ BestModelIndexL3 ]]
BestModelFormulaL3<-Level3Res$Model[[BestModelIndexL3]]

BestModel<-BestModelL3
BestModelDeletions<-arrange(dropterm(BestModel),desc(AIC))
```

```

# order by terms that increase AIC the most when they are the one dropped
BestModelDeletions

## Single term deletions
##
## Model:
## preserved ~ CumErr + stimlen + log_freq
##           Df Deviance   AIC
## CumErr    1   2864.0 2870.0
## stimlen    1   2815.4 2821.4
## log_freq   1   2807.2 2813.2
## <none>      1   2805.1 2813.1

#####
# Single deletions from best model
#####

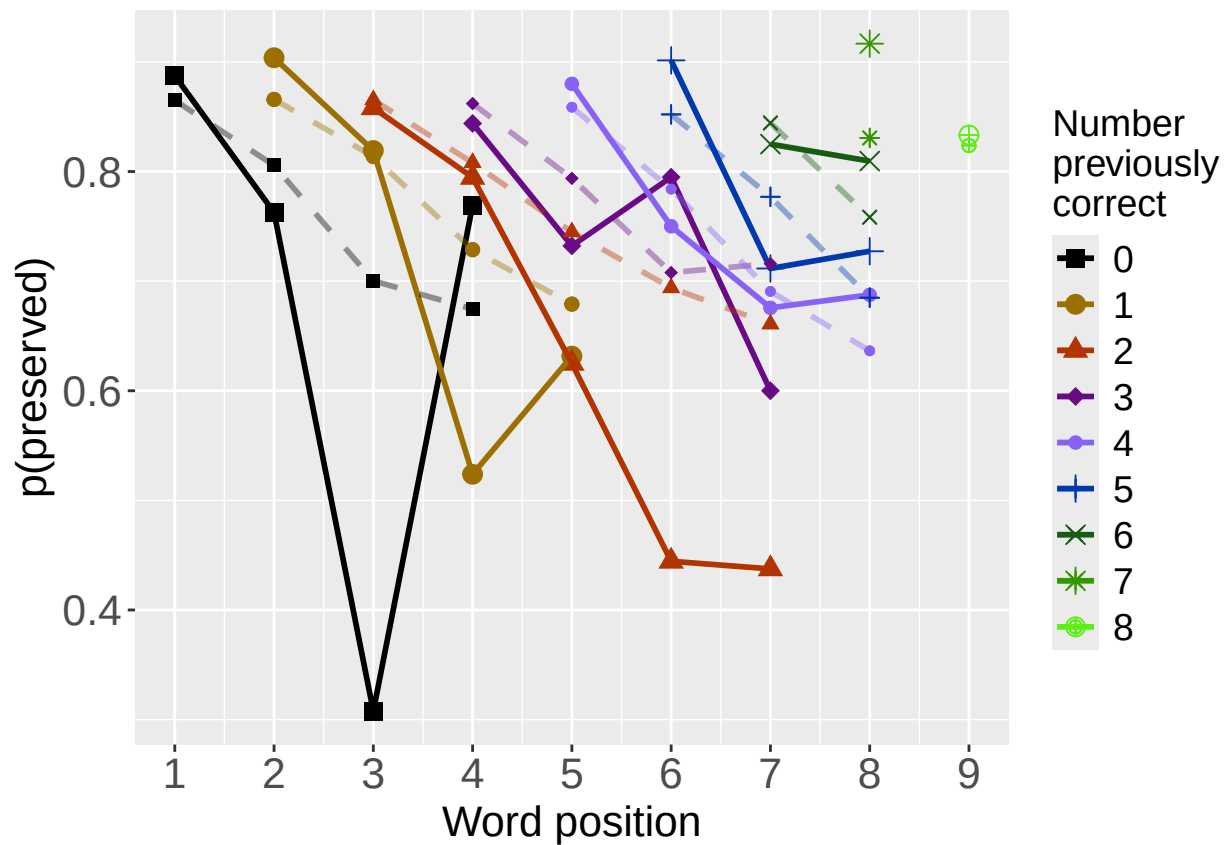
write.csv(BestModelDeletions,paste0(TablesDir,CurPat,"_",
                                   RunLabel,"_",CurTask,
                                   "_best_model_single_term_deletions.csv"),
          row.names = TRUE)

# plot prev err and prev cor with predicted values
PosDat$OAPred<-fitted(BestModel)

PrevCorPlot<-PlotObsPredPreviousCorrect(PosDat,"preserved","OAPred",palette_values,shape_values)

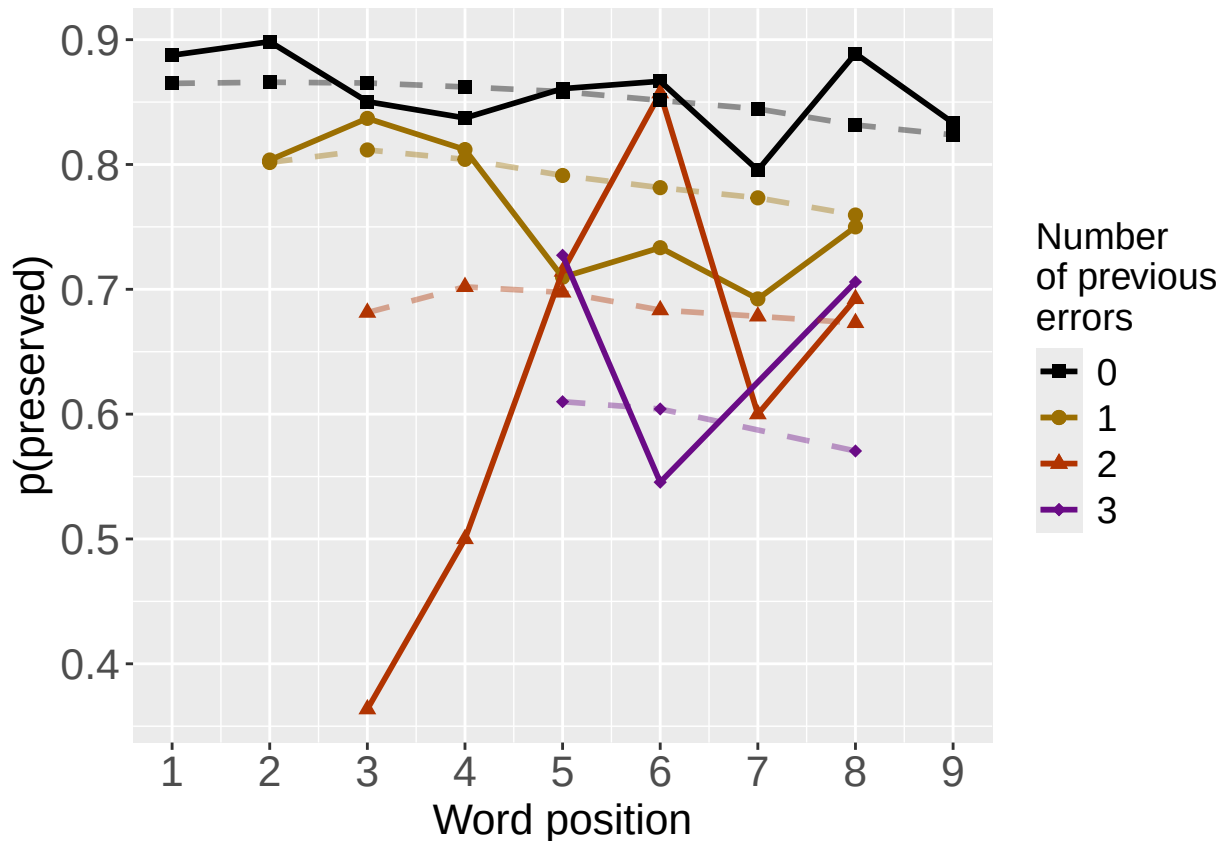
## `summarise()` has grouped output by 'CumPres'. You can override using the `groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `groups` argument.
print(PrevCorPlot)

```



```
PrevErrPlot<-PlotObsPredPreviousError(PosDat,"preserved","OAPred",palette_values,shape_values)

## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
print(PrevErrPlot)
```



```

PlotName<-paste0(FigDir,CurPat,"_",
                  RunLabel,"_",CurTask,
                  "_ObsPred_BestFullModel")
ggsave(filename=paste(PlotName,"_prev_correct.tif",sep=""),plot=PrevCorPlot,device="tiff",compression =

## Saving 6.5 x 4.5 in image
ggsave(filename=paste(PlotName,"_prev_error.tif",sep=""),plot=PrevErrPlot,device="tiff",compression = "

## Saving 6.5 x 4.5 in image
if(DoSimulations){
  BestModelFormulaL3Rnd <- BestModelFormulaL3
  if(grepl("CumPres",BestModelFormulaL3Rnd)){
    BestModelFormulaL3Rnd <- gsub("CumPres","RndCumPres",BestModelFormulaL3Rnd)
  }
  if(grepl("CumErr",BestModelFormulaL3Rnd)){
    BestModelFormulaL3Rnd <- gsub("CumErr","RndCumErr",BestModelFormulaL3Rnd)
  }

  RndModelAIC<-numeric(length=RandomSamples)
  for(rindex in seq(1,RandomSamples)){
    # Shuffle cumulative values
    PosDat$RndCumPres <- ShuffleWithinWord(PosDat,"CumPres")
    PosDat$RndCumErr <- ShuffleWithinWord(PosDat,"CumErr")
    BestModelRnd <- glm(as.formula(BestModelFormulaL3Rnd),
                       family="binomial",data=PosDat)
    RndModelAIC[rindex] <- BestModelRnd$aic
  }
}

```

```

}
ModelNames<-c(paste0("***",BestModelFormulaL3),
              rep(BestModelFormulaL3Rnd,RandomSamples))
AICValues <- c(BestModelL3$aic,RndModelAIC)
BestModelRndDF <- data.frame(Name=ModelNames,AIC=AICValues)
BestModelRndDF <- BestModelRndDF %>% arrange(AIC)
BestModelRndDF <- rbind(BestModelRndDF,
                        data.frame(Name=c("Random average"),
                                   AIC=c(mean(RndModelAIC))))
BestModelRndDF <- rbind(BestModelRndDF,
                        data.frame(Name=c("Random SD"),
                                   AIC=c(sd(RndModelAIC))))
write.csv(BestModelRndDF,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                                "_best_model_with_random_cum_term.csv"),
          row.names = FALSE)
}

# plot influence factors
num_models <- nrow(BestModelDeletions) - 1
# minus 1 because last
# model is always <none> and we don't want to include that

if(num_models > 4){
  last_model_num <- 4
}else{
  last_model_num <- num_models
}

FinalModelSet<-ModelSetFromDeletions(BestModelFormulaL3,
                                     BestModelDeletions,
                                     last_model_num)

PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_FactorPlots")

FactorPlot<-PlotModelSetWithFreq(PosDat,FinalModelSet,
                                 palette_values,FinalModelSet,PptID=CurPat)

## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##          data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##          1.832      -0.507
##
## Degrees of Freedom: 3200 Total (i.e. Null); 3199 Residual
## Null Deviance: 2902
## Residual Deviance: 2820 AIC: 2824
## log likelihood: -1410.114
## Nagelkerke R2: 0.04237641
## % pres/err predicted correctly: -869.1616
## % of predictable range [ (model-null)/(1-null) ]: 0.02935871

```

```

## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      stimlen
##      2.6260      -0.4443      -0.1092
##
## Degrees of Freedom: 3200 Total (i.e. Null); 3198 Residual
## Null Deviance:      2902
## Residual Deviance: 2807 AIC: 2813
## log likelihood: -1403.602
## Nagelkerke R2: 0.04901653
## % pres/err predicted correctly: -865.4967
## % of predictable range [ (model-null)/(1-null) ]: 0.03344677
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      stimlen      log_freq
##      2.55881      -0.44506      -0.09989      0.03877
##
## Degrees of Freedom: 3200 Total (i.e. Null); 3197 Residual
## Null Deviance:      2902
## Residual Deviance: 2805 AIC: 2813
## log likelihood: -1402.524
## Nagelkerke R2: 0.05011264
## % pres/err predicted correctly: -864.8858
## % of predictable range [ (model-null)/(1-null) ]: 0.03412825
## *****

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.

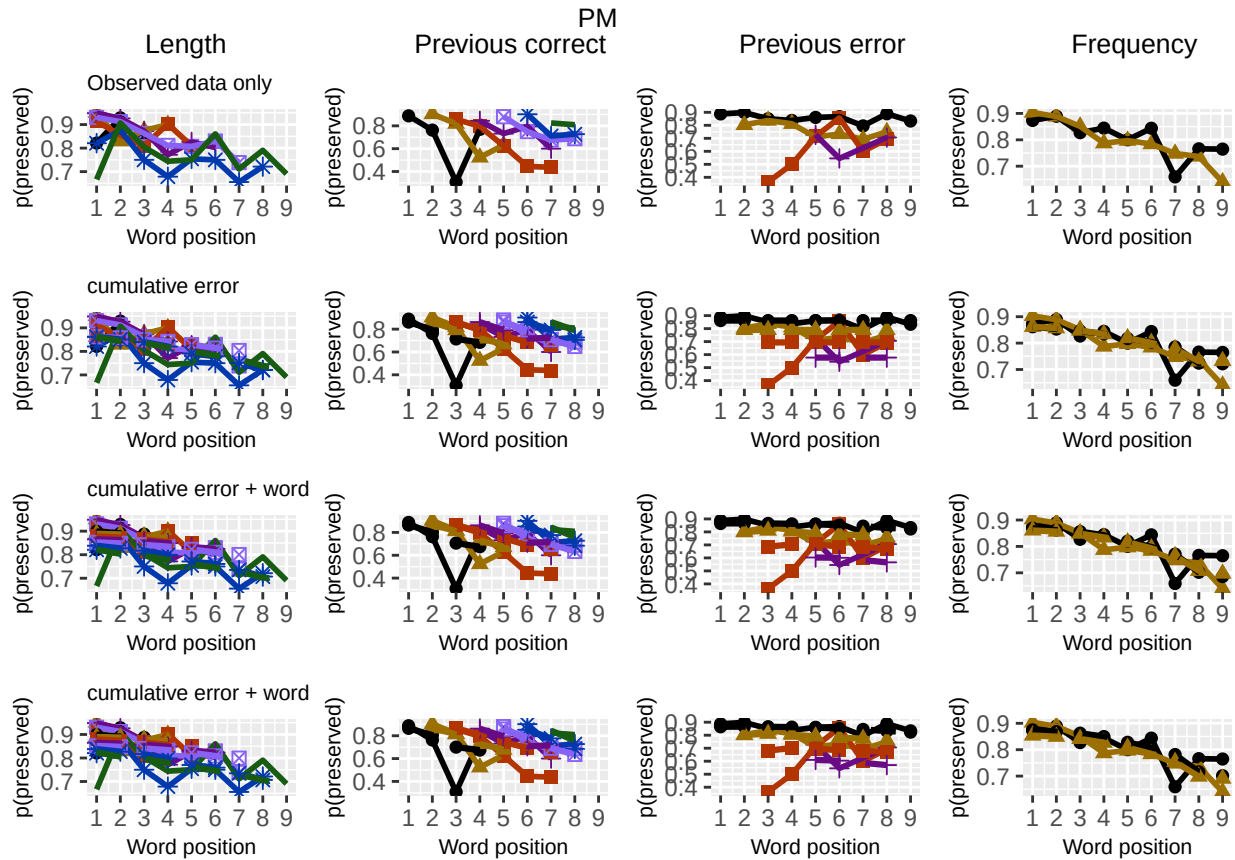
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.

```

```

## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 4 rows containing missing values or values outside the scale range (`geom_point()`).
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).
## Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 4 rows containing missing values or values outside the scale range (`geom_point()`).
## Removed 4 rows containing missing values or values outside the scale range (`geom_point()`).
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).
## Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 4 rows containing missing values or values outside the scale range (`geom_point()`).
## Removed 4 rows containing missing values or values outside the scale range (`geom_point()`).
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).
## Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 4 rows containing missing values or values outside the scale range (`geom_point()`).
## Removed 4 rows containing missing values or values outside the scale range (`geom_point()`).
ggsave(paste0(PlotName, ".tif"), plot=FactorPlot, width = 360, height=400, units="mm", device="tiff", compress=
# use \blandscape and \elandscape to make markdown plots landscape if needed
FactorPlot

```



```

DA.Result<-dominanceAnalysis(BestModel)
DAContributionAverage<-ConvertDominanceResultToTable(DA.Result)
write.csv(DAContributionAverage,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                                     CurTask,"_dominance_analysis_table.csv"),
          row.names = TRUE)
kable(DAContributionAverage)

```

	CumErr	stimlen	log_freq
McFadden	0.0241067	0.0078703	0.0014746
SquaredCorrelation	0.0214975	0.0070538	0.0013217
Nagelkerke	0.0214975	0.0070538	0.0013217
Estrella	0.0218937	0.0071441	0.0013385

	deviance	deviance_explained
CumErr + stimlen + log_freq	2805.048	97.08097
CumErr + stimlen	2807.203	94.92571
CumErr	2820.228	81.90040
null	2902.129	0.00000

	percent_explained	percent_of_explained_deviance	increment_in_explained
CumErr + stimlen + log_freq	3.345164	100.00000	2.220065
CumErr + stimlen	3.270899	97.77994	13.416960
CumErr	2.822080	84.36298	84.362975
null	0.000000	NA	0.000000

```
deviance_differences_df<-GetDevianceSet(FinalModelSet,PosDat)
write.csv(deviance_differences_df,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                                         CurTask,"_deviance_differences_table.csv"),
          row.names = FALSE)
deviance_differences_df
```

```
##                                model deviance deviance_explained percent_explained percent_of_explained_deviance
## CumErr + stimlen + log_freq CumErr + stimlen + log_freq 2805.048          97.08097          3.345164          100.00000
## CumErr + stimlen              CumErr + stimlen 2807.203          94.92571          3.270899          97.77994
## CumErr                        CumErr 2820.228          81.90040          2.822080          84.36298
## null                          null 2902.129          0.00000          0.000000          NA
##                                increment_in_explained
## CumErr + stimlen + log_freq          2.220065
## CumErr + stimlen                    13.416960
## CumErr                              84.362975
## null                               0.000000
```

```
kable(deviance_differences_df[,2:3], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

```
kable(deviance_differences_df[,c(4:6)], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

```
NagContributions <- t(DAContributionAverage[3,])
NagTotal<-sum(NagContributions)
```

```

NagPercents <- NagContributions / NagTotal
NagResultTable <- data.frame(NagContributions = NagContributions, NagPercents = NagPercents)
names(NagResultTable)<-c("NagContributions","NagPercents")
write.csv(NagResultTable,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                                "_nagelkerke_contributions_table.csv"),row.names = TRUE)
NagPercents

```

```

##           Nagelkerke
## CumErr    0.7196288
## stimlen   0.2361263
## log_freq  0.0442449

```

```

sse_results_list<-compare_SS_accounted_for(FinalModelSet,"preserved ~ 1",PosDat,N_cutoff=10)

```

```

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.

## Warning in (cumerr_residuals^2) * (N_values$cumerr_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumcor_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.

## Warning in (cumerr_residuals^2) * (N_values$cumerr_N): longer object length is not a multiple of shorter object length

```

model	p_accounted_for	model_deviance
preserved ~ CumErr	0.8153224	2820.228
preserved ~ CumErr+stimlen	0.8234223	2807.203
preserved ~ CumErr+stimlen+log_freq	0.8260690	2805.048

```
## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumcor_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.

## Warning in (cumerr_residuals^2) * (N_values$cumerr_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumcor_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
```

```
sse_table <- sse_results_table(sse_results_list)
write.csv(sse_table, paste0(TablesDir, CurPat, "_", RunLabel, "_",
                           CurTask, "_sse_results_table.csv"), row.names = TRUE)
sse_table
```

```
##               model p_accounted_for model_deviance diff_CumErr diff_CumErr+stimlen diff_CumErr+stimlen+log_freq
## 1      preserved ~ CumErr      0.8153224      2820.228 0.000000000      -0.008099875      -0.01074653
## 2      preserved ~ CumErr+stimlen      0.8234223      2807.203 0.008099875      0.000000000      -0.00264665
## 3 preserved ~ CumErr+stimlen+log_freq      0.8260690      2805.048 0.010746525      0.002646650      0.00000000
```

```
kable(sse_table[,1:3], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

```
write.csv(results_report_DF, paste0(TablesDir, CurPat, "_", RunLabel, "_",
                                   CurTask, "_results_report_df.csv"), row.names = FALSE)
```

```
ncols_in_table <- ncol(sse_table)
kable(sse_table[,c(1,4:ncols_in_table)], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

model	diff_CumErr	diff_CumErr+stimlen	diff_CumErr+stimlen+log_freq
preserved ~ CumErr	0.0000000	-0.0080999	-0.0107465
preserved ~ CumErr+stimlen	0.0080999	0.0000000	-0.0026466
preserved ~ CumErr+stimlen+log_freq	0.0107465	0.0026466	0.0000000